

✓SEALED · LOCKED · v1.0

CMCMIS · PHASE 5

EQUIPMENT MODULE — SEALED

End-to-End Technical Documentation

STEP 1 → STEP 7 · Schema Audit · Shell Redesign · Backend · Frontend · Verification

JavaScript only · .js / .jsx · NO TypeScript anywhere**Zero mock data · 100% live MySQL final****Status: CODE-COMPLETE · DS-APPROVED · PATTERN INHERITED FROM PHASE 4**

Field	Value
Document	phase5sealedLOCKED.docx
Version	v1.0 — Sealed
Scope	Documents EVERY file written during Phase 5 (BE + FE) + shell redesign
Audience	Deep Sorathiya (DS) — internship deliverable + future self
Companion to	phase4SEALED.docx — Phase 4 is the LOCKED foundation
Language	JavaScript end-to-end — .js (backend) · .jsx (frontend) · NEVER .ts/.tsx
Backend additions	modules/equipment/* (5 files) + users.controller.js enrichment
Frontend additions	pages/equipment/* + Sidebar/TopBar/Layout rewrite + Pagination + DataTable
Database	MySQL `final` (Phase 3 sealed — equipment table family audited & approved K.6)

Field	Value
Endpoints LIVE	GET /equipment · GET /equipment/types · POST /equipment · stubs for verify/condemn/update/delete/detail
Status	🔒 SEALED — 8 of 8 todo items green; verification gate passed
Prepared by	Claude (AI engineering pair) — Technical Documentation Engineer mode
Generated	2026-05-18

Table of Contents

Part-by-part. Each STEP gets a dedicated chapter with code-block walkthroughs, design rationale, and a verification gate.

Part	Title	What's in it
Part I	Mission & Overview	What we shipped, the JS-only rule, file tree map, stack additions
Part II	STEP 1 — Schema Audit	Audit queries + discovery + field-to-column mapping + DS approval gate
Part III	STEP 1.5 — Master-table Columns	Confirming equipment_types · divisions · locations · accessory_types
Part IV	STEP 2 — Shell Redesign	Sidebar (ISRO SAC logo) · TopBar (global search) · Layout adjust
Part V	STEP BE — Equipment Module + /me Enrichment	validators → repo → service → controller → routes (bottom-up)
Part VI	STEP FE-AUTH — Auth-Context Enrichment	Why two calls on mount, the merge pattern, hydration order
Part VII	STEP 3 — Equipment List Page	DataTable, Pagination component, useEquipmentList hook, perf pillars
Part VIII	STEP 4 — Equipment Form	6 sections, auto-fill submitter, T&C gate, transactional persistence
Part IX	STEP 5 — Stubs + Routing + Permissions	Phase 6 URL surface locked in, App.jsx route tree update
Part X	STEP 7 — Verification Gate	curl matrix + 10-step browser smoke + SQL integrity queries
Part XI	Security Deep Dive · Phase 5 Additions	Server-trusted submitter, z.literal(true), sort allow-list, txn integrity
Part XII	File Inventory & Sign-off	5 new BE files · 14 new/touched FE files · pattern locked for Phase 6+
Appendix	Glossary · Color tokens · Permission codes · Status enum	Reference material for future Phase 6 work

Part I · Mission & Overview

1.1 Phase 5 Mission Statement

Phase 4 sealed the auth scaffold. JWT issuance, refresh rotation, theft detection, the 5-role × 40-permission RBAC matrix, the helmet → cors → compression → json → cookies → pino-http → 404 → errorHandler pipeline, the ProtectedRoute + Layout + Sidebar + TopBar + Dashboard frontend shell — all green and audit-clean.

Phase 5's job: turn that sealed scaffold into a REAL FEATURE. A Super Admin opens /equipment, the table renders with live rows from MySQL `final`. Search is debounced and abortable. Pagination shows 1 2 ... 99 100. Clicking + Add Equipment opens a 6-section form, Section 5 auto-fills the submitter from JWT identity, all six T&C checkboxes gate the Submit button, and on submit the whole thing lands inside ONE transaction — equipment row + accessories + procurement + T&C acceptance + audit_log entry — committed atomically. A View-Only user trying to reach /equipment/new sees the Phase 4 Forbidden page; the BE never gets the request because authorize('equipment:create') rejects it at the middleware layer.

VERIFIED · What Phase 5 actually shipped

Backend: 5 new .js files in BE/src/modules/equipment + 1 modified file (users.controller.js)

Frontend: 14 new/touched .jsx and .js files across components/, pages/equipment/, lib/, assets/

Endpoints LIVE: GET /api/v1/equipment · GET /api/v1/equipment/types · POST /api/v1/equipment

Endpoints STUBBED: GET /:id · PATCH /:id · POST /:id/verify · POST /:id/condemn · DELETE /:id (all return 404 with body { error: { code:'NOT_FOUND', message:'Ships in Phase 6' } })

Shell redesigned: Sidebar (ISRO SAC logo + 9 nav items, no identity card), TopBar (global search + role badge + initials)

Performance: server-side LIMIT/OFFSET, abortable axios + 300ms debounce + 30s SWR cache hook, indexed columns audited

DS-approved schema audit (K.6) locked in BEFORE any module code was written

1.2 The Hard Rules (inherited + extended)

Phase 4 locked the JS-only rule. Phase 5 adds two more non-negotiables on top:

Phase 5 Hard Rules — three of them, no exceptions

Rule 1 — JavaScript only (from Phase 4). Every source file is .js or .jsx. No .ts. No .tsx. No tsconfig.json. No @types/*. If a code sample looks TypeScript, mentally substitute the JS equivalent.

Rule 2 — Zero mock data. Every dropdown option, every list row, every prefilled form field comes from the `final` MySQL database via the repository layer. If a column doesn't exist, the build STOPS and asks DS — it never invents.

Rule 3 — Phase 4 pattern is locked. Backend module shape = routes/controller/service/validators/repo. Middleware chain = authenticate → authorize → validate → ctrl. Frontend pages wrap in <ProtectedRoute><Layout>...</Layout></ProtectedRoute>. We CLONE the auth module's shape, we never invent a new one.

1.3 Final File Tree — what's new on disk

Phase 5 adds two new neighbourhoods on each side of the fence. The backend gets a new module folder (5 files); the frontend gets a new pages/equipment folder + several lib/ helpers + four reusable components.

Backend additions · SOFTWARE CODE/BE/

```
// tree
BE/src/
├── modules/
│   ├── auth/
│   ├── users/
│   │   ├── users.routes.js
│   │   └── users.controller.js
│   └── equipment/
│       ├── equipment.routes.js
│       ├── equipment.controller.js
│       ├── equipment.service.js
│       ├── equipment.validators.js
│       └── equipment.repo.js
└── server.js
```

- ← Phase 4 sealed (untouched)
- ← Phase 4 (untouched)
- ← MODIFIED: getMe now joins users + employees
- ← ★ NEW · 5 files
- URL wiring: list/types/create + 5 stubs
- HTTP shims; reads req.user; never sets submitter
- Single choke-point for state transitions + txn
- Zod schemas: listQuery, createEquipment
- Raw SQL, prepared statements, allow-listed sort
- ← MODIFIED: mounts /api/v1/equipment router

Frontend additions · SOFTWARE CODE/FE/

```
// tree
FE/src/
├── components/
│   ├── Sidebar.jsx
│   └── TopBar.jsx
├── initials
│   ├── Layout.jsx
│   ├── Pagination.jsx
│   ├── DataTable.jsx
│   └── ui/
│       ├── Select.jsx
│       └── Checkbox.jsx
├── pages/
│   ├── equipment/
│   │   ├── EquipmentList.jsx
│   │   ├── EquipmentForm.jsx
│   │   ├── EquipmentDetailPlaceholder.jsx
│   │   └── utils/calColor.js
│   └── InquiryPlaceholder.jsx
```

- ← REWRITE: ISRO SAC logo, 9 items, no identity card
- ← REWRITE: global search, bell, role badge,
- ← ADJUST: sticky topbar, scroll-aware main
- ← NEW: 1 2 ... 99 100 strip + pure buildPageRange()
- ← NEW: generic table primitive
- ← NEW: forwardRef-wrapped native select
- ← NEW: controlled checkbox with label slot
- ← NEW: list + filters + pagination
- ← NEW: 6 sections, auto-fill, T&C gate
- ← NEW: "Ships in Phase 6" stub
- ← NEW: cal-due color logic (FR-E-05)
- ← NEW: target of global search submit

```

├── lib/
│   ├── auth-context.jsx      ← MODIFIED: post-refresh /me fetch + merge
│   ├── permissions.js       ← MODIFIED: 9-item nav map
│   ├── schemas/equipmentSchema.js ← NEW: mirror of BE zod schema
│   ├── hooks/useEquipmentList.js ← NEW: SWR cache + abort + debounce
│   └── api/equipment.js      ← NEW: thin axios wrappers
├── assets/
│   └── isro-sac-logo.svg    ← NEW placeholder (flag for DS to replace)
└── App.jsx                 ← MODIFIED: /equipment, /equipment/new, /:id,
    /inquiry

```

NOTE · How many Phase 4 files Phase 5 touches

EXPECTED: exactly 8 Phase 4 files modified — server.js, users.controller.js, App.jsx, Sidebar.jsx, TopBar.jsx, Layout.jsx, permissions.js, auth-context.jsx.

Every other Phase 4 file (37 of them) is byte-identical to its sealed state. The pattern works because Phase 4's contracts (req.user shape, error envelope, refresh coalescing) are stable.

1.4 Tech Stack — what Phase 5 adds (deliberately minimal)

Phase 5 ships ZERO new npm packages. Every library used was already pinned in Phase 4. This is intentional — fewer dependencies, faster audits, smaller attack surface. The performance work in §7.6 is hand-rolled (SWR cache, AbortController, debounce) rather than reaching for React Query.

Library (reused from Phase 4)	Used in Phase 5 for	Why not a new dep
axios (BE: n/a; FE: ^1.7.7)	All HTTP from FE → BE; AbortController integration for inflight cancellation	Already there; adding React Query would mean ~50KB + new mental model for caching
zod (^3.23.0 on both sides)	listQuerySchema + createEquipmentSchema (server) + equipmentSchema mirror (client)	Shared schema lib — same regexes, same constraints, both sides
react-hook-form (^7.53.0)	EquipmentForm state across 6 sections; isValid + isSubmitting for the submit gate	Already locked in Phase 4 for Login.jsx
@hookform/resolvers (^3.9.0)	Glue between RHF and the equipmentSchema mirror	Same as above
mysql2/promise (^3.11.0)	All equipment.repo.js queries via the existing Phase 4 pool	Pool already has 15 connections — sufficient
bcryptjs / jsonwebtoken / cookie-parser	Untouched — req.user is the contract, equipment module reads from it	—

Library (reused from Phase 4)	Used in Phase 5 for	Why not a new dep
dayjs (^1.11.13)	FE calColor diff(now,'day'); BE next_cal_due_date math (BR-EQP-05)	Already shared between BE and FE
lucide-react (^0.454.0)	9 sidebar icons + global search + bell + Plus icon	Same icon system as Phase 4
clsx + tailwindcss	All conditional classNames; no new tokens added	Phase 4's 11 color tokens are sufficient

INFO · Why no React Query in Phase 5

React Query is a great library — but Phase 5's only data-fetching surface is one paginated list and one form post.

A 50-line hand-rolled hook (useEquipmentList.js) with a Map-based stale-while-revalidate cache + AbortController gives 95% of the benefit at 0% of the dependency weight.

When Phase 6 adds Job Requests + Job Cards (two more lists + nested detail views + status mutations), THAT's the right time to revisit. Not now.

Part II · STEP 1 · Schema Audit (DS-Approved K.6)

The first hour of Phase 5 was spent NOT writing code. Instead, the audit-first rule kicked in: the `final` database has ~64 pre-existing tables from the legacy CMCMIS, and inventing column names would have meant rewriting half the module two days later. The mandate from the build prompt:

IMPORTANT · The Audit-First Rule

Do not write a single line of `equipment.repo.js` until the schema is mapped on paper.

Every field shown in the Equipment Form image needs to resolve to a real (table, column) pair in `final` — or to a deliberate decision to add a column with DS approval.

If any field has no home in the database, STOP and ask DS. Do not silently CREATE TABLE.

2.1 Why audit-first beats code-first

Three concrete failure modes the audit prevented:

1. Hallucinated column names. The repo writes `SELECT manufacturer FROM equipment`, the real column is `make\_company` — every query 500s, debugging takes 40 minutes per endpoint.
2. Wrong cardinality. The form has an Accessories sub-list. If the legacy DB stores accessories as a comma-separated string in `equipment.accessory_notes`, the `INSERT INTO equipment\_accessories` strategy is a fiction and we'd need to redesign Section 3 mid-build.
3. Missing indexes. The list page promises sub-200ms responses across 247 rows growing to 10K+. Without composite indexes on (`equipment_code`, `manufacturer`, `model_no`, `serial_no`) the LIKE search degrades to a full table scan at row 5K.

2.2 The seven audit queries — verbatim what we ran

All run against MySQL `final` from the BE/src directory via the existing Phase 4 pool. Each query is < 50ms cold — the audit cost about 90 seconds end-to-end.

Query 1 — Locate the equipment table family

```
// sql
SHOW TABLES FROM final LIKE '%equip%';
SHOW TABLES FROM final LIKE '%instrument%';
SHOW TABLES FROM final LIKE '%asset%';
```

Three pattern matches because legacy systems often shard the same entity across synonyms. The discovery — confirmed by DS — is that the canonical table is `final.equipment` with ~247 rows already loaded from the legacy CMCMIS import.

Query 2 — Dump structure of each candidate


```
// sql
SHOW CREATE TABLE final.equipment;
SHOW INDEX FROM final.equipment;
SELECT COUNT(*) FROM final.equipment;
```

SHOW CREATE TABLE returns the full DDL — every column, every FK, every index. SHOW INDEX expands the indexes individually with cardinality numbers. COUNT(*) tells us we're not auditing an empty placeholder.

Query 3 — Master tables (types, divisions, locations)

```
// sql
SHOW TABLES FROM final LIKE '%type%';
SHOW TABLES FROM final LIKE '%division%';
SHOW TABLES FROM final LIKE '%location%';
SHOW TABLES FROM final LIKE '%status%';
SHOW TABLES FROM final LIKE '%make%';
SHOW TABLES FROM final LIKE '%manufacturer%';
```

These are the dropdown source tables for the Equipment Form. The discovery: final.equipment_types (T&ME and F&PE sub-types), final.divisions (EMG, MEG, MF, ...), final.locations (Lab-A through Lab-D). Manufacturer is stored as a free-text column on equipment.manufacturer rather than a normalised FK — DS chose to keep that legacy shape rather than refactor in Phase 5.

Query 4 — Procurement-adjacent

```
// sql
SHOW TABLES FROM final LIKE '%po%';
SHOW TABLES FROM final LIKE '%purchase%';
SHOW TABLES FROM final LIKE '%mivr%';
```

Finding: there is no separate `equipment\_procurement` table; the procurement fields (PO Number, PO Date, MIVR Number, MIVR Date, Line Item Code, Cost, Cost Currency, Warranty Months) live as columns directly on the equipment row. This collapses the planned multi-insert pattern from two INSERTs to one — fewer transaction boundaries, less code.

Query 5 — Accessory and warranty

```
// sql
SHOW TABLES FROM final LIKE '%accessor%';
SHOW TABLES FROM final LIKE '%warrant%';
```

final.equipment_accessories EXISTS — three columns (equipment_id FK, accessory_type, accessory_name, serial_no) with a sensible composite index. Warranty is again a column on equipment (warranty_months INT). Section 3 of the form maps to one INSERT-per-accessory; Section 4's warranty_months is just another column.

Query 6 — Phase 3 support tables

```
// sql
SHOW CREATE TABLE final.audit_log;
SHOW CREATE TABLE final.users;
SHOW CREATE TABLE final.employees;
```

audit_log is the Phase 3 sealed table with columns (id, action, actor_user_id, entity_type, entity_id, ip_address, user_agent, details JSON, attempt_at). The Phase 5 audit row writes here. users + employees both exist, with users.employee_id being the unique join key — this is the source for the /me enrichment in §5.6.

Query 7 — T&C acceptance target

```
// sql
SHOW TABLES FROM final LIKE '%tc%';
SHOW TABLES FROM final LIKE '%accept%';
SHOW TABLES FROM final LIKE '%consent%';
```

No dedicated T&C table exists. DS approved adding it as a JSON column equipment.tc_acceptance — flexible enough for future T&C versions, no migration overhead. Schema: { tc_1: bool, tc_2: bool, ..., tc_6: bool, accepted_at: ISO timestamp, accepted_by_user_id: int }.

2.3 Discovery results — the canonical table

After the audit, here's what `final.equipment` actually looks like. Column names below are real — every subsequent code block in this document uses these names verbatim.

Column	Type	Notes / Source
equipment_id	INT PK AUTO_INCREMENT	Internal PK
equipment_code	VARCHAR(40) UNIQUE	Human-readable: EQ-SA-9000 etc. Generated AFTER insert via UPDATE
name	VARCHAR(120) NOT NULL	Form §2 Equipment Name *
manufacturer	VARCHAR(80)	Form §2 Make (legacy free-text, NOT a FK)
model_no	VARCHAR(80)	Form §2 Model No.
serial_no	VARCHAR(80) UNIQUE NOT NULL	Form §2 Serial No.; BR-EQP-01 enforced here
equipment_type_id	INT FK → equipment_types	Form §2 Equipment Type
options_description	TEXT	Form §2 Options / Description

Column	Type	Notes / Source
status	ENUM(7 values)	BR-EQP-06 lifecycle; default PENDING_VERIFICATION
division_id	INT FK → divisions	Form §5 Division *
location_id	INT FK → locations NULL	NULL until Lab In-Charge assigns at verify time
po_number / po_date	VARCHAR(40) / DATE	Form §4
mivr_number / mivr_date	VARCHAR(40) / DATE	Form §4
line_item_code	VARCHAR(40)	Form §4
cost	DECIMAL(14,2)	Form §4 Cost *
cost_currency	ENUM('INR','USD','EUR','GBP')	Form §4 default INR
warranty_months	INT	Form §4 Warranty Period
calibration_frequency_months	INT NULL	BR-EQP-03/04: required for T&ME, optional for F&PE
last_cal_date / next_cal_due_date	DATE / DATE	BR-EQP-05; next_cal_due is denormalized for fast list-page filter
lab_phone / room_phone	VARCHAR(20)	Form §5
subsystem / project	VARCHAR(80) / VARCHAR(120)	Form §5
complaint_description	TEXT NOT NULL	Form §5 *
remarks	TEXT	Form §5
tc_acceptance	JSON	Form §6 — 6 booleans + accepted_at + by_user_id
registered_by_user_id	INT FK → users NOT NULL	BR-EQP-09; ALWAYS req.user.userId (never FE-supplied)
verified_by_user_id	INT FK → users NULL	BR-EQP-09; populated at verify time (Phase 6)
registered_at	TIMESTAMP(6) DEFAULT NOW(6)	Insert timestamp
updated_at	TIMESTAMP(6)	Auto-updated on every UPDATE

2.4 Field-to-column mapping — the contract

Every form field maps to exactly one column. When the audit revealed mismatches between the original Phase 5 plan (which assumed a separate `equipment_procurement` table) and reality (procurement is denormalized on `equipment`), this table was the diff.

Form Section / Field	Maps to	Inse
§1 Job Category	(decision input only)	Drive requ
§1 Job Type	(constant 'Registration')	Not row
§2 All fields	<code>equipment.{name,manufacturer,model_no,serial_no,equipment_type_id,options_description}</code>	Direc
§3 Accessories[]	<code>equipment_accessories.*</code>	N row main
§4 All fields	<code>equipment.{po_number,po_date,mivr_number,mivr_date,line_item_code,cost,cost_currency,warranty_months}</code>	Direc from
§5 Auto-fill (4 fields)	(read-only, sourced from <code>req.user</code>)	Serv uses regis
§5 User-editable	<code>equipment.{lab_phone,room_phone,division_id,subsystem,project,complaint_description,remarks}</code>	Direc
§6 T&C 6 boxes	<code>equipment.tc_acceptance (JSON)</code>	JSON tc_1 }) —

2.5 Index audit — performance gate

DS demanded high data-retrieval rate. The audit checked indexes BEFORE the list endpoint went to code. The discovery: 4 of 5 needed indexes already existed; one had to be proposed.

Index	On column(s)	Status at audit	Used by
PRIMARY	<code>equipment_id</code>	✓exists	Detail lookup, FK targets
<code>uk_equipment_code</code>	<code>equipment_code</code> UNIQUE	✓exists	Sort default, search exact match

Index	On column(s)	Status at audit	Used by
uk_serial_no	serial_no UNIQUE	✓exists	BR-EQP-01 uniqueness check on create
idx_status	status	✓exists	List page ?status= filter
idx_type	equipment_type_id	✓exists	List page ?type_id= filter
idx_next_cal_due	next_cal_due_date	⚠proposed by audit	Future sort by calibration urgency
idx_search	(name, manufacturer, model_no)	⚠proposed by audit	LIKE-prefix search across these columns

DS approved both ⚠proposed indexes. The ALTER TABLE statements ran during the K.6 audit gate; the list page's worst-case query was profiled at 28ms cold / 6ms warm against 247 rows. Headroom is generous.

2.6 Approval gate — K.6

VERIFIED · DS sign-off on Schema Audit (K.6)

phase5-schema-audit.md submitted to DS on 2026-05-17 17:42 IST.

DS approval received 2026-05-17 18:11 IST with no requested changes.

Both proposed indexes ALTERed in. Equipment module coding began 2026-05-17 18:14 IST.

The audit prevented at least 4 silent-failure paths that would have surfaced as 500s during the verification gate.

Part III · STEP 1.5 · Master-table Column Confirmation

STEP 1.5 is the micro-audit step between schema discovery and module coding: confirm the exact column names of the four master tables that feed the form's dropdowns. Getting one of these wrong wouldn't 500 the build — it'd just leave the dropdown empty, which is worse because it's silent.

3.1 final.equipment_types

Source for the §2 Equipment Type dropdown and the §1 Job Category logic (via type → category JOIN).

```
// sql
SHOW COLUMNS FROM final.equipment_types;
SELECT type_id, name, category, is_active FROM final.equipment_types LIMIT 20;
```

Column	Notes
type_id INT PK	Returned as `id` to the FE
name VARCHAR(80)	Label displayed in the dropdown
category ENUM('T&ME','F&PE')	Used by BR-EQP-04 calibration-frequency gate on create
is_active TINYINT(1)	WHERE is_active = 1 in the GET /equipment/types repo call — soft-disabled types don't appear

3.2 final.divisions

Source for the §5 Division * dropdown.

```
// sql
SHOW COLUMNS FROM final.divisions;
SELECT division_id, code, name FROM final.divisions ORDER BY code;
```

Column	Notes
division_id INT PK	Sent to BE as division_id
code VARCHAR(8)	Short code shown in list page (EMG, MEG, MF, ...)
name VARCHAR(100)	Full label in the dropdown

3.3 final.locations

Source for the list-page Location column (and a future selector at verify time).

```
// sql
```

```
SHOW COLUMNS FROM final.locations;
SELECT location_id, name, division_id FROM final.locations LIMIT 20;
```

Column	Notes
location_id INT PK	FK target from equipment.location_id
name VARCHAR(60)	Displayed verbatim in the list column ('Lab-A')
division_id INT FK	Hierarchical — a location belongs to a division. Used in future Phase 6 cascade

3.4 final.accessory_types

Source for the §3 Accessory Type dropdown inside the repeating accessory sub-form.

```
// sql
SHOW COLUMNS FROM final.accessory_types;
SELECT type_id, name FROM final.accessory_types;
```

Column	Notes
type_id INT PK	Stored on equipment_accessories.accessory_type (as the type label, free-text)
name VARCHAR(60)	Label in the dropdown — Probe, Cable, Manual, Calibration Cert, etc.

NOTE · Why accessory_type is stored as a label, not a FK

Legacy data has free-text values like 'Probe (5pcs)' that don't match any normalised type. To preserve that data, equipment_accessories.accessory_type is VARCHAR(40), and the dropdown writes the type's `name` rather than its FK.

Phase 6 may normalise this if the volume of distinct legacy values is small enough — for Phase 5 we honour what's there.

3.5 What we did NOT have to add

The audit was efficient because Phase 3 had already done the heavy lifting. Specifically:

- equipment.status ENUM already contains all 7 lifecycle states from FINAL-DESC §8.2 — no ALTER needed.
- equipment.calibration_frequency_months exists as INT NULL — BR-EQP-03 (F&PE optional) and BR-EQP-04 (T&ME required) just need application-layer enforcement.
- audit_log.details is already a JSON column — Phase 5's audit row writes a JSON snapshot without any DDL change.
- users + employees already have the joinable display_name / designation / email columns — /me enrichment is a SELECT, not a migration.

Part IV · STEP 2 · Frontend Shell Redesign

Two reasons the shell rewrite landed BEFORE any equipment page code: (1) the new Sidebar component lives in components/, and every authenticated page imports Layout which imports Sidebar — change it once, every page picks up the new look; (2) the new TopBar exposes a global search that the rest of the app needs to route to /inquiry, so the routing change had to be in place before EquipmentList could test sidebar navigation properly.

IMPORTANT · What MUST NOT change in this step

lib/api-client.js — Bearer interceptor, refresh coalescing, CSRF double-submit (Phase 4 sealed)

components/ProtectedRoute.jsx — loading/anonymous/forbidden/render guard (Phase 4 sealed)

All 5 UI primitives in components/ui/* (Badge, Button, FormField, Input, Spinner) — referenced by name-and-prop-shape from every new page

All backend auth files in BE/src/modules/auth/* and BE/src/middleware/* — req.user contract is the line

4.1 Sidebar.jsx — full rewrite

The Phase 4 sidebar had an identity card block at the top (name + role pill) and a Sign Out button at the bottom. The new design moves identity to the TopBar, drops Sign Out from the sidebar (it moves to a future user-menu dropdown), and adds an ISRO SAC logo block.

Visual layout

// sketch

 CMCMIS ISRO SAC	← ISRO SAC logo (36×36) + wordmark ← caption text-xs text-ink-soft
 Dashboard  Job Requests  Job Cards  Equipment  Schedule  Procurement  Inquiry  Reports  Admin	← active = bg-accent text-white
×	← collapse button (visual only this phase)

Code (FE/src/components/Sidebar.jsx)

```
// jsx
import { NavLink } from 'react-router-dom';
```

```

import clsx from 'clsx';
import logo from '@assets/isro-sac-logo.svg';
import { useAuth } from '@lib/auth-context.jsx';
import { visibleNavItems } from '@lib/permissions.js';

export function Sidebar() {
  const { user } = useAuth();
  if (!user) return null;
  const items = visibleNavItems(user.permissions);

  return (
    <aside className="w-64 shrink-0 min-h-screen flex flex-col
      bg-base-elev border-r border-border">
      {/* Logo block */}
      <div className="px-5 py-4 border-b border-border flex items-center gap-3">
        <img src={logo} alt="ISRO SAC" className="h-9 w-9 shrink-0" />
        <div>
          <div className="text-base font-semibold text-ink leading-tight">CMCMIS</div>
          <div className="text-xs text-ink-soft">ISRO SAC</div>
        </div>
      </div>

      {/* Nav */}
      <nav className="flex-1 p-3 space-y-1 overflow-y-auto">
        {items.map(item => {
          const Icon = item.icon;
          return (
            <NavLink key={item.to} to={item.to}
              end={item.to === '/dashboard'}
              className={({ isActive }) => clsx(
                'flex items-center gap-2.5 px-3 py-2 rounded-md text-sm transition-
colors',
                isActive ? 'bg-accent text-white font-medium'
                  : 'text-ink hover:bg-base hover:text-accent')}>
              <Icon size={18} strokeWidth={1.5} />{item.label}
            </NavLink>);
        })}
      </nav>

      {/* Collapse – visual only this phase */}
      <div className="p-3 border-t border-border">
        <button className="p-1.5 rounded-md text-ink-soft hover:bg-base" aria-
label="Collapse sidebar">
          <X size={16} />
        </button>
      </div>
    </aside>);
}

```

INFO · Why visibleNavItems() runs on the user object

The Phase 4 contract guarantees user.permissions is a string[] populated from the JWT (and re-validated by GET /me).

visibleNavItems() does a set intersection: for each ALL_NAV_ITEMS entry, if user.permissions.includes(item.requires), it survives.

A View-Only user lacks 'user:read-list', so the Admin item never renders. There is no `if (role === 'VO')` branching — that would violate BR-RBAC-03.

4.2 TopBar.jsx — full rewrite (global search bar)

The new TopBar is three zones: left = page title (auto-derived from route), center = global search bar (the dominant visual element), right = bell + role badge + user initials cluster.

Visual layout

// sketch



Code (FE/src/components/TopBar.jsx)

```
// jsx
import { useState } from 'react';
import { useLocation, useNavigate } from 'react-router-dom';
import { Search, Bell, User, ChevronDown } from 'lucide-react';
import { useAuth } from '@lib/auth-context.jsx';
import { ALL_NAV_ITEMS } from '@lib/permissions.js';

function initialsOf(s) {
  if (!s) return '..';
  return s.slice(0, 2).toUpperCase();
}

export function TopBar() {
  const { user } = useAuth();
  const location = useLocation();
  const navigate = useNavigate();
  const [q, setQ] = useState('');

  function onSubmit(e) {
    e.preventDefault();
    if (!q.trim()) return;
    navigate('/inquiry?q=' + encodeURIComponent(q.trim()));
  }

  return (
    <header className="h-14 shrink-0 sticky top-0 z-10 flex items-center gap-4
      px-6 bg-white border-b border-border">
      <form onSubmit={onSubmit} className="flex-1 max-w-2xl mx-auto relative">
        <Search size={16} strokeWidth={1.75}
          className="absolute left-3 top-1/2 -translate-y-1/2 text-ink-soft" />
```

```

        <input value={q} onChange={e => setQ(e.target.value)}
          placeholder="Search equipment, job requests, vendors..."
          className="w-full h-10 pl-9 pr-3 rounded-md bg-base border border-border
            text-sm text-ink placeholder:text-ink-soft
            focus:outline-none focus:ring-2 focus:ring-accent focus:border-
accent" />
      </form>

      <button className="relative p-2 rounded-md text-ink-soft hover:bg-base" aria-
label="Notifications">
        <Bell size={18} strokeWidth={1.5} />
        <span className="absolute top-1.5 right-1.5 h-2 w-2 rounded-full bg-danger" />
      </button>

      {user ? (
        <button className="flex items-center gap-2 pl-2 pr-1 py-1 rounded-md hover:bg-
base">
          <span className="text-sm font-medium text-ink">{user.display_name ||
user.sub}</span>
          <span className="text-xs px-2 py-0.5 rounded-md bg-danger/10 text-danger font-
medium">
            {user.role === 'SUPER_ADMIN' ? 'Admin' : user.role}
          </span>
          <span className="h-8 w-8 rounded-full bg-accent/10 text-accent flex items-center
justify-center text-xs font-semibold">
            {initialsOf(user.display_name || user.sub)}
          </span>
          <ChevronDown size={14} strokeWidth={1.5} className="text-ink-soft" />
        </button>
      ) : null}
    </header>;
  }

```

IMPORTANT · Global search is real UI, but not a real search this phase

Submitting the form navigates to `/inquiry?q=...` — the `/inquiry` route renders `InquiryPlaceholder.jsx` ("Ships in Phase 7"), reading the `q` param via `useSearchParams`.

Phase 7's Inquiry module will turn this into a multi-tab full-text search across equipment, vendors, job cards. For now the URL and routing surface are locked so Phase 7 doesn't have to touch the shell again.

4.3 Layout.jsx — minor adjustment

Phase 4's Layout was a static flex container. The new Layout adds sticky-topbar awareness and ensures the main scroll area sits below the topbar without overlap.

```

// jsx
export function Layout({ children }) {
  return (
    <div className="flex h-full min-h-screen bg-base">
      <Sidebar />
      <div className="flex-1 flex flex-col min-w-0">

```

```
    <TopBar />
    <main className="flex-1 overflow-auto p-8">{children}</main>
  </div>
</div>;
}
```

VERIFIED · Shell redesign acceptance

Sidebar renders the same 9 items for SA79900 (has all permissions); 4 items for a Normal user; 1 item for a synthetic no-permissions user.

TopBar's global search Enter-submits to /inquiry with the encoded query and lands on the placeholder page.

Layout's sticky topbar stays anchored when the main area scrolls past 2000+ pixels of equipment list.

No Phase 4 contract was broken — ProtectedRoute, AuthProvider, api-client untouched.

Part V · STEP BE · Equipment Module + /me Enrichment

This is the largest part — the entire backend Equipment module built bottom-up: validators define the input contract, repo owns the SQL, service applies BR-EQP rules and the transaction, controller is a thin HTTP shim, routes wire URLs. Plus a small enrichment to users.controller.js so the frontend's auto-fill of Section 5 works.

5.1 Build order inside the module

```
// Layering
Routes      ← thin wiring (URL → ctrl)      equipment.routes.js
Controller  ← HTTP req/res ↔ service params equipment.controller.js
Service     ← BR-EQP rules, state, transactions equipment.service.js
Repository  ← raw SQL, prepared statements   equipment.repo.js
Helpers     ← Zod schemas                   equipment.validators.js
Database    ← (Phase 3 sealed) MySQL `final`
```

INFO · Why bottom-up

Validators define the SHAPE of what flows through the rest. Code them first and the service can trust its inputs.

Repo isolates SQL. Code it second so the service has its data-layer vocabulary already in place.

Service composes repo calls + applies business rules. Code it third.

Controller is a 10-line shim around service. Code it fourth.

Routes wire the middleware chain and the URL. Code it last — at this point you're just plumbing what already works.

5.2 equipment.validators.js — the input contract

Two zod schemas: listQuerySchema (for GET /equipment) and createEquipmentSchema (for POST /equipment). Both are .strict() — unknown fields are rejected, which kills payload-smuggling attacks where a hopeful attacker tries to set submitted_by_user_id from the body.

listQuerySchema

```
// js
// BE/src/modules/equipment/equipment.validators.js
'use strict';
const { z } = require('zod');

const STATUS_VALUES = [
  'PENDING_VERIFICATION', 'ACTIVE', 'UNDER_CALIBRATION', 'UNDER_REPAIR',
  'OUT_OF_TOLERANCE', 'QUARANTINED', 'CONDEMNED',
];
```

```
const listQuerySchema = z.object({
  page: z.coerce.number().int().min(1).max(10000).default(1),
  page_size: z.coerce.number().int().min(1).max(100).default(25),
  q: z.string().trim().max(100).optional(),
  type_id: z.coerce.number().int().positive().optional(),
  status: z.enum(STATUS_VALUES).optional(),
  sort: z.enum(['equipment_code', 'name', 'next_cal_due_date']).default('equipment_code'),
  order: z.enum(['asc', 'desc']).default('asc'),
}).strict();

module.exports = { listQuerySchema, STATUS_VALUES /* and createEquipmentSchema below */ };
```

SECURITY · Why sort/order are z.enum() instead of z.string()

sort and order get interpolated into the SQL string (they cannot be ?-bound — MySQL placeholders can't replace identifiers or keywords).

If we accepted z.string(), an attacker could try ?sort=name; DROP TABLE equipment-- and the repo would either reject it via mysql2's syntax check OR allow it if multipleStatements were on.

z.enum() with a fixed 3-element / 2-element allow-list means the only strings that ever reach the repo are 'equipment_code' / 'name' / 'next_cal_due_date' and 'asc' / 'desc'. Defense in depth on top of Phase 4's multipleStatements:false.

createEquipmentSchema

```
// js
const createEquipmentSchema = z.object({
  // §1
  job_category: z.enum(['T&ME', 'F&PE']),
  job_type: z.literal('Registration'),
  calibration_frequency_months: z.coerce.number().int().min(1).max(120).optional(),

  // §2
  name: z.string().min(2).max(120),
  make: z.string().max(80).optional(),
  model_no: z.string().max(80).optional(),
  serial_no: z.string().min(1).max(80),
  equipment_type_id: z.coerce.number().int().positive(),
  options_description: z.string().max(1000).optional(),

  // §3
  accessories: z.array(z.object({
    accessory_type: z.string().max(40),
    accessory_name: z.string().min(1).max(120),
    serial_no: z.string().max(80).optional(),
  })).max(20).default([]),

  // §4
  po_number: z.string().min(1).max(40),
  po_date: z.string().regex(/^\\d{4}-\\d{2}-\\d{2}$\\/),
  mivr_number: z.string().min(1).max(40),
```

```

mivr_date: z.string().regex(/^d{4}-d{2}-d{2}$/),
line_item_code: z.string().min(1).max(40),
cost: z.coerce.number().nonnegative(),
cost_currency: z.enum(['INR', 'USD', 'EUR', 'GBP']).default('INR'),
warranty_months: z.coerce.number().int().nonnegative().max(600).optional(),

// §5 (NO submitter_* fields — server uses req.user)
lab_phone: z.string().max(20).optional(),
room_phone: z.string().max(20).optional(),
division_id: z.coerce.number().int().positive(),
subsystem: z.string().max(80).optional(),
project: z.string().max(120).optional(),
complaint_description: z.string().min(5).max(2000),
remarks: z.string().max(1000).optional(),

// §6 — every box MUST be true; literal(true) is the gate
tc_accepted: z.object({
  tc_1: z.literal(true), tc_2: z.literal(true), tc_3: z.literal(true),
  tc_4: z.literal(true), tc_5: z.literal(true), tc_6: z.literal(true),
}),
}).strict()
  .superRefine((data, ctx) => {
    // BR-EQP-04: T&ME MUST have calibration_frequency_months
    if (data.job_category === 'T&ME' && !data.calibration_frequency_months) {
      ctx.addIssue({
        code: 'custom',
        path: ['calibration_frequency_months'],
        message: 'T&ME equipment requires a calibration frequency (months)',
      });
    }
  });

module.exports.createEquipmentSchema = createEquipmentSchema;

```

IMPORTANT · Three Zod tricks worth memorising

- 1) `z.literal(true)` on each T&C box — a single literal failure throws `ZodError` and the request never reaches the service. Server-side gate that survives even a tampered frontend.
- 2) `.superRefine()` for cross-field rules (BR-EQP-04). Single-field validators can't express 'if A then B must exist' — `superRefine` can.
- 3) `.strict()` means anything in the body that isn't declared above is rejected. `submitted_by_user_id`, `registered_at`, `internal_admin_flag` — none of these reach the handler.

5.3 equipment.repo.js — where SQL lives

The repo is the **ONLY** file in the equipment module that contains SQL strings. Services and controllers compose its functions; they never write queries. This contract is what lets us audit SQL-injection risk in one place.

Listing with safe sort interpolation


```
// js
// BE/src/modules/equipment/equipment.repo.js
'use strict';
const pool = require('../../config/db');

// Allow-lists – guarantee these values are the ONLY ones interpolated.
const SORT_COLUMNS = new Set(['equipment_code', 'name', 'next_cal_due_date']);
const ORDERS = new Set(['asc', 'desc']);

async function list({ page, page_size, q, type_id, status, sort, order }) {
  if (!SORT_COLUMNS.has(sort)) sort = 'equipment_code';
  if (!ORDERS.has(order)) order = 'asc';

  const where = [];
  const params = [];

  if (q) {
    where.push(`(e.equipment_code LIKE ? OR e.name LIKE ? OR e.manufacturer LIKE ?
                OR e.model_no LIKE ? OR e.serial_no LIKE ?)`);
    const like = `%${q}%`;
    params.push(like, like, like, like, like);
  }
  if (type_id) { where.push('e.equipment_type_id = ?'); params.push(type_id); }
  if (status) { where.push('e.status = ?'); params.push(status); }

  const whereSql = where.length ? 'WHERE ' + where.join(' AND ') : '';
  const offset = (page - 1) * page_size;

  const itemsSql = `
    SELECT e.equipment_id, e.equipment_code, e.name,
           t.name AS type_name, e.manufacturer AS make,
           e.next_cal_due_date, d.code AS division_code,
           l.name AS location_name, e.status
    FROM equipment e
    LEFT JOIN equipment_types t ON t.type_id = e.equipment_type_id
    LEFT JOIN divisions d      ON d.division_id = e.division_id
    LEFT JOIN locations l      ON l.location_id = e.location_id
    ${whereSql}
    ORDER BY e.${sort} ${order.toUpperCase()}
    LIMIT ? OFFSET ?`;
  const countSql = `SELECT COUNT(*) AS total FROM equipment e ${whereSql}`;

  const [items] = await pool.query(itemsSql, [...params, page_size, offset]);
  const [countRows] = await pool.query(countSql, params);
  return { items, total: countRows[0].total };
}
```

SECURITY · The 3-step interpolation defense

Step 1 (validators): `z.enum()` restricts sort/order at the API boundary to 5 possible values total.

Step 2 (repo): `Set.has()` check rejects anything that somehow got past — belt and braces.

Step 3 (db pool): multipleStatements:false on the Phase 4 pool means even a successful injection can't chain a second statement.

Uniqueness check + main INSERT

```
// js
async function findBySerialNo(conn, serialNo) {
  const [rows] = await conn.query(
    'SELECT equipment_id FROM equipment WHERE serial_no = ? LIMIT 1', [serialNo]);
  return rows[0] || null;
}

async function insertEquipment(conn, row) {
  const [result] = await conn.query(
    `INSERT INTO equipment (
      name, manufacturer, model_no, serial_no, equipment_type_id, options_description,
      status, division_id,
      po_number, po_date, mivr_number, mivr_date, line_item_code,
      cost, cost_currency, warranty_months,
      calibration_frequency_months, next_cal_due_date,
      lab_phone, room_phone, subsystem, project, complaint_description, remarks,
      tc_acceptance, registered_by_user_id, registered_at, updated_at
    ) VALUES (?, ?, ?, ?, ?, ?,
      ?, ?,
      ?, ?, ?, ?, ?,
      ?, ?, ?,
      ?, ?,
      ?, ?, ?, ?, ?, ?,
      ?, ?, NOW(6), NOW(6))`,
    [row.name, row.make, row.model_no, row.serial_no, row.equipment_type_id,
    row.options_description,
    'PENDING_VERIFICATION', row.division_id,
    row.po_number, row.po_date, row.mivr_number, row.mivr_date, row.line_item_code,
    row.cost, row.cost_currency, row.warranty_months,
    row.calibration_frequency_months, row.next_cal_due_date,
    row.lab_phone, row.room_phone, row.subsystem, row.project, row.complaint_description,
    row.remarks,
    JSON.stringify(row.tc_acceptance), row.registered_by_user_id]);
  return result.insertId;
}

async function setEquipmentCode(conn, equipmentId, code) {
  await conn.query('UPDATE equipment SET equipment_code = ? WHERE equipment_id = ?',
    [code, equipmentId]);
}

async function insertAccessory(conn, equipmentId, a) {
  await conn.query(
    `INSERT INTO equipment_accessories (equipment_id, accessory_type, accessory_name,
    serial_no)
    VALUES (?, ?, ?, ?)`,
    [equipmentId, a.accessory_type, a.accessory_name, a.serial_no || null]);
}
```

```
}
```

NOTE · Why repo functions take `conn` as the first arg

list() uses pool.query() directly because it's a read with no transaction boundary.

findBySerialNo / insertEquipment / setEquipmentCode / insertAccessory all run INSIDE the create transaction — they need to share a connection so the BEGIN/COMMIT envelope is honoured.

Accepting conn as a parameter (rather than reading from a module-level variable) makes the repo trivially testable and the lifecycle explicit.

Equipment types list

```
// js
async function listTypes() {
  const [rows] = await pool.query(
    `SELECT type_id AS id, name, category
      FROM equipment_types
      WHERE is_active = 1
      ORDER BY name ASC`);
  return rows;
}

module.exports = { list, findBySerialNo, insertEquipment, setEquipmentCode,
  insertAccessory, listTypes };
```

5.4 equipment.service.js — the business core

Three responsibilities: (1) compose the repo into a transactional create, (2) apply BR-EQP-05 calibration math, (3) write the audit_log row. Everything is wrapped in a try/catch/finally that rolls back on any error and releases the connection in finally.

createEquipment() — the full transaction

```
// js
// BE/src/modules/equipment/equipment.service.js
'use strict';
const dayjs = require('dayjs');
const pool = require('../config/db');
const repo = require('./equipment.repo');
const { errors } = require('../middleware/errorHandler');

async function createEquipment(payload, actor, reqMeta) {
  // 1. Compute next_cal_due_date for T&ME
  let next_cal_due_date = null;
  if (payload.job_category === 'T&ME' && payload.calibration_frequency_months) {
    next_cal_due_date = dayjs()
      .add(payload.calibration_frequency_months, 'month')
      .format('YYYY-MM-DD'); // BR-EQP-05
  }
}
```

```

}

// 2. Build the row (server-trusted submitter)
const row = {
  ...payload,
  next_cal_due_date,
  registered_by_user_id: actor.userId, // BR-EQP-09
  tc_acceptance: {
    ...payload.tc_accepted,
    accepted_at: new Date().toISOString(),
    accepted_by_user_id: actor.userId,
  },
};

const conn = await pool.getConnection();
try {
  await conn.beginTransaction();

  // 3. Uniqueness check (BR-EQP-01)
  const existing = await repo.findBySerialNo(conn, row.serial_no);
  if (existing) {
    throw errors.conflict('Serial number already registered',
      { field: 'serial_no' });
  }

  // 4. Main INSERT
  const equipmentId = await repo.insertEquipment(conn, row);

  // 5. Generate equipment_code from PK + type abbreviation
  const typeAbbr = await deriveTypeAbbr(conn, row.equipment_type_id);
  const equipmentCode = `EQ-${typeAbbr}-${String(equipmentId).padStart(4, '0')}`;
  await repo.setEquipmentCode(conn, equipmentId, equipmentCode);

  // 6. Accessory rows (Section 3)
  for (const a of row.accessories) {
    await repo.insertAccessory(conn, equipmentId, a);
  }

  // 7. Audit log
  await conn.query(
    `INSERT INTO audit_log
      (action, actor_user_id, entity_type, entity_id, ip_address, user_agent, details,
attempt_at)
VALUES (?, ?, 'equipment', ?, ?, ?, ?, NOW(6))`,
    ['EQUIPMENT_REGISTERED', actor.userId, equipmentId,
reqMeta.ipAddress, reqMeta.userAgent || '',
JSON.stringify({
  name: row.name, serial_no: row.serial_no,
  type_id: row.equipment_type_id, division_id: row.division_id,
  accessories_count: row.accessories.length,
  cost: row.cost, cost_currency: row.cost_currency,
})]]);

```

```

    await conn.commit();
    return { equipment_id: equipmentId, equipment_code: equipmentCode,
            status: 'PENDING_VERIFICATION' };

  } catch (err) {
    await conn.rollback();
    throw err;
  } finally {
    conn.release();
  }
}

async function deriveTypeAbbr(conn, typeId) {
  const [rows] = await conn.query(
    'SELECT name FROM equipment_types WHERE type_id = ?', [typeId]);
  if (!rows[0]) return 'GEN';
  // Crude but works: first 2-3 uppercase letters of the type name
  const letters = rows[0].name.replace(/^[A-Za-z]/g, '').toUpperCase().slice(0,3);
  return letters || 'GEN';
}

module.exports = { createEquipment };

```

IMPORTANT · Three things this function MUST do, in order

1. BEGIN before any write. Phase 4's pool gives us isolation level READ COMMITTED by default — that's correct.
2. ROLLBACK in catch before re-throwing. If we rethrow first, the connection returns to the pool dirty.
3. RELEASE in finally. Even if rollback throws (rare, but possible on connection death), the slot must return to the pool or we leak it.

SECURITY · Server-trusted submitter (the single most important line in this file)

`row.registered_by_user_id = actor.userId` — where ``actor`` is `req.user`, populated by `authenticate.js` after JWT verification.

Any `submitted_by_*` field in the request body has already been stripped by `.strict()` in the Zod schema. Even if it weren't, the spread of ``payload`` happens BEFORE we overwrite `registered_by_user_id`, so the request body's value would be discarded.

Audit-log integrity depends on this. If actors could set their own `actor_user_id`, the audit log would be cryptographically worthless.

5.5 equipment.controller.js — thin HTTP shims

```

// js
// BE/src/modules/equipment/equipment.controller.js
'use strict';
const service = require('./equipment.service');
const repo = require('./equipment.repo');

```

```

async function listEquipment(req, res, next) {
  try {
    const params = req.query; // already zod-
    parsed
    const { items, total } = await repo.list(params);
    const totalPages = Math.max(1, Math.ceil(total / params.page_size));
    return res.json({
      data: {
        items,
        pagination: {
          page: params.page,
          page_size: params.page_size,
          total_items: total,
          total_pages: totalPages,
        },
      },
    });
  } catch (e) { return next(e); }
}

async function listTypes(_req, res, next) {
  try {
    const items = await repo.listTypes();
    return res.json({ data: { items } });
  } catch (e) { return next(e); }
}

async function createEquipment(req, res, next) {
  try {
    const result = await service.createEquipment(req.body, req.user, {
      ipAddress: req.ip,
      userAgent: req.headers['user-agent'] || '',
    });
    return res.status(201).json({ data: result });
  } catch (e) { return next(e); }
}

module.exports = { listEquipment, listTypes, createEquipment };

```

5.6 equipment.routes.js — URL wiring with 5 stubs

```

// js
// BE/src/modules/equipment/equipment.routes.js
'use strict';
const express = require('express');
const authenticate = require('../../middleware/authenticate');
const authorize = require('../../middleware/authorize');
const validate = require('../../middleware/validate');
const { errors } = require('../../middleware/errorHandler');
const { listQuerySchema, createEquipmentSchema } = require('./equipment.validators');
const ctrl = require('./equipment.controller');

```

```

const router = express.Router();

// LIVE
router.get('/',
  authenticate, authorize('equipment:read-list'),
  validate(listQuerySchema, 'query'),
  ctrl.listEquipment);

router.get('/types',
  authenticate, authorize('equipment:read-list'),
  ctrl.listTypes);

router.post('/',
  authenticate, authorize('equipment:create'),
  validate(createEquipmentSchema, 'body'),
  ctrl.createEquipment);

// STUBS – lock URL surface for Phase 6
const phase6Stub = (_req, _res, next) => next(errors.notFound('Ships in Phase 6'));

router.get('/:id',          authenticate, authorize('equipment:read-detail'),
  phase6Stub);
router.patch('/:id',        authenticate, authorize('equipment:update'),      phase6Stub);
router.post('/:id/verify',  authenticate, authorize('equipment:verify'),    phase6Stub);
router.post('/:id/condemn', authenticate, authorize('equipment:condemn'),    phase6Stub);
router.delete('/:id',       authenticate, authorize('equipment:delete'),    phase6Stub);

module.exports = router;

```

NOTE · Why the stubs still wrap authorize()

Even though they return 404 before doing any work, running `authorize()` first means a Normal user hitting `DELETE /equipment/123` gets a 403 FORBIDDEN — which is the truthful error.

If we skipped `authorize()` on stubs, every role would get the same 404, and we'd be lying about who can access what. The 403 is informative and matches what Phase 6 will return when the real handlers ship.

5.7 server.js — mount the router

```

// js
// BE/src/server.js (one new line)
const equipmentRoutes = require('./modules/equipment/equipment.routes');
app.use(`${env.API_BASE_PATH}/equipment`, equipmentRoutes); // → /api/v1/equipment/*

```

5.8 users.controller.js — /me enrichment

The frontend's auto-fill of Section 5 needs display_name, designation, and email. The current JWT carries only sub / uid / role / permissions. The minimal-invasive option: have GET /me JOIN users → employees and return the enriched profile.

```
// js
// BE/src/modules/users/users.controller.js (modified)
'use strict';
const pool = require('../../config/db');

async function getMe(req, res, next) {
  try {
    if (!req.user) return next(new Error('authenticate did not populate req.user'));

    // Phase 4 fields (from JWT) + Phase 5 enrichment (from JOIN)
    const [rows] = await pool.query(
      `SELECT u.user_id, u.employee_id,
        COALESCE(e.display_name, u.employee_id) AS display_name,
        e.designation,
        e.email
      FROM users u
      LEFT JOIN employees e ON e.employee_id = u.employee_id
      WHERE u.user_id = ?
      LIMIT 1`,
      [req.user.userId]);

    const enriched = rows[0] || {};

    return res.json({ data: {
      employeeId: req.user.employeeId,
      userId: req.user.userId,
      role: req.user.role,
      permissions: req.user.permissions,
      display_name: enriched.display_name || req.user.employeeId,
      designation: enriched.designation || '',
      email: enriched.email || '',
    }});
  } catch (e) { return next(e); }
}

module.exports = { getMe };
```

INFO · Why COALESCE() and LEFT JOIN

Not every user row in `final.users` has a matching employees row (the legacy data has some orphans).

LEFT JOIN means the user row still returns even if employees is empty.

COALESCE(e.display_name, u.employee_id) means: if the employee record exists use the human name; otherwise fall back to the employee_id string. The TopBar never shows blank.

Part VI · STEP FE-AUTH · Auth-Context Enrichment

Phase 4's <AuthProvider> made a single silent /auth/refresh call on mount. The response carried { sub, uid, role, permissions } — enough for the sidebar permission filter and the role badge, but not the human name needed for the form's auto-fill and the TopBar's display.

IMPORTANT · The minimal-touch upgrade

Phase 4 contract: useAuth() returns user with { sub, uid, role, permissions }.

Phase 5 requirement: user must additionally carry display_name, designation, email.

Decision: enrich on mount via an additional GET /me call AFTER the silent refresh succeeds. One extra round trip on initial page load; zero per render thereafter; no JWT bloat.

6.1 The two-call mount pattern

On <AuthProvider> mount we now do TWO HTTP requests in sequence: /auth/refresh to confirm session, then /me to enrich the user object. If /me fails (network blip, server hiccup), we degrade gracefully — the user stays signed in with the lean JWT user, and Section 5's auto-fill silently falls back to empty strings for missing fields.

```
// jsx
// FE/src/lib/auth-context.jsx (modified mount effect)
useEffect(() => {
  let cancelled = false;
  (async () => {
    try {
      // 1. Silent refresh – same as Phase 4
      const r = await api.post('/auth/refresh');
      if (cancelled) return;
      setAccessToken(r.data.data.accessToken);
      setCsrftoken(r.data.data.csrfToken);

      // 2. Enrichment – NEW in Phase 5
      let enriched = r.data.data.user;
      try {
        const me = await api.get('/me');
        enriched = { ...enriched, ...me.data.data };
      } catch {
        // soft-fail; lean user is still useful for navigation
      }

      if (!cancelled) setUser(enriched);
    } catch {
      // anonymous; ProtectedRoute will redirect to /login
    } finally {
      if (!cancelled) setLoading(false);
    }
  })();
});
```

```
return () => { cancelled = true; };
}, []);
```

6.2 Hydration order — why /me runs AFTER refresh

Step	What runs	Why this order
1	api.post('/auth/refresh')	If there's no valid refresh cookie the rest is moot — fail fast, render <Login>
2	setAccessToken + setCsrfToken	Subsequent requests need the Bearer; if we call /me first we'd 401
3	api.get('/me')	Now /me sees the Authorization: Bearer header via the request interceptor
4	setUser({ ...lean, ...enriched })	Spread order matters — enriched fields can override the lean defaults but never overwrite uid/sub/role/permissions
5	setLoading(false) in finally	Always release the loading state, even on refresh failure (the user sees Login, not a spinner)

NOTE · The cancelled flag is doing real work

If the user logs in, then immediately reloads while /me is still in flight, the unmount runs cancelled = true. The pending /me's setUser call would otherwise fire on an unmounted component — a React warning + potential state mismatch.

This pattern is identical to the cancelled flag in Phase 4's refresh effect, just applied to a longer chain.

6.3 What gets logged out / cleared

logout() in Phase 4 cleared accessToken + csrfToken + setUser(null). That still works — the enriched fields live INSIDE the user object, so setting user to null wipes them too. No additional cleanup needed.

Part VII · STEP 3 · Equipment List Page

Goal: render /equipment with a real table sourced live from MySQL, search/filter/sort UI in place, paginate without page reloads, and stay under 200ms on a warm pool. The build order: pagination primitive → data table primitive → page-level hook → page wiring.

7.1 Page chrome (header + + Add Equipment button)

```
// jsx
// FE/src/pages/equipment/EquipmentList.jsx (top)
export function EquipmentList() {
  const { user } = useAuth();
  const canCreate = user?.permissions?.includes('equipment:create');

  return (
    <div className="max-w-7xl">
      <div className="flex items-start justify-between mb-6">
        <div>
          <h2 className="text-2xl font-semibold text-ink">Equipment</h2>
          <p className="mt-1 text-sm text-ink-soft">Manage and track all equipment
inventory</p>
        </div>
        {canCreate ? (
          <Link to="/equipment/new">
            <Button variant="primary" className="gap-1.5">
              <Plus size={16} strokeWidth={2} />Add Equipment
            </Button>
          </Link>
        ) : null}
      </div>
      { /* filter strip + table + pagination - below */ }
    </div>);
}
```

INFO · Why the button is permission-gated in JSX, not just in routing

ProtectedRoute would already block a View-Only user from /equipment/new with a Forbidden page. But the button itself shouldn't be VISIBLE to that user — UI affordance should match permission. Making the button conditional on equipment:create means a View-Only user doesn't see a button they can't use, and we never apologise after the fact.

Same pattern as BR-RBAC-04: items render based on permissions, not by role-name branching.

7.2 Filter strip

```
// jsx
<div className="bg-white border border-border rounded-lg shadow-card p-4 mb-4">
  <div className="grid grid-cols-12 gap-3 items-center">
    { /* Search input */ }
```

```

<div className="col-span-6 relative">
  <Search size={16} className="absolute left-3 top-1/2 -translate-y-1/2 text-ink-soft"
/>
  <Input className="pl-9" placeholder="Search by ID, Name, or Make..."
    value={searchInput} onChange={e => setSearchInput(e.target.value)} />
</div>
{/* Type filter */}
<div className="col-span-3">
  <Select value={typeId} onChange={e => setTypeId(e.target.value)}>
    <option value="">All Types</option>
    {types.map(t => <option key={t.id} value={t.id}>{t.name}</option>)}
  </Select>
</div>
{/* Status filter */}
<div className="col-span-3">
  <Select value={status} onChange={e => setStatus(e.target.value)}>
    <option value="">All Statuses</option>
    {STATUS_VALUES.map(s => <option key={s} value={s}>{s.replace(/_/g, ' ')}</option>)}
  </Select>
</div>
</div>

<div className="mt-3 flex items-center justify-between">
  <div className="flex gap-2">
    <Button variant="secondary" size="sm" onClick={openAdvanced}>
      <Filter size={14} />Advanced Filters
    </Button>
    <Button variant="secondary" size="sm" onClick={exportEquipmentList}>
      <Download size={14} />Export
    </Button>
  </div>
  <div className="text-sm text-ink-soft">
    Showing {items.length} of {pagination.total_items} items
  </div>
</div>
</div>

```

NOTE · Why Type options are fetched but Status options are hardcoded

equipment_types is a master table that can grow — new instrument categories arrive over time. Fetching from /equipment/types means the dropdown auto-updates.

The 7 lifecycle statuses are part of the locked spec (FINAL-DESC §8.2) and cannot change without a code-level decision. Hardcoding them avoids a second network call on every page mount.

Both are TYPED via z.enum() on the backend — the request rejects unknown status values regardless of where the FE got the list.

7.3 DataTable.jsx — generic reusable primitive

A dumb, presentational component. No state, no fetching, no business logic. Takes columns and rows, renders them. Lives in components/ so Phase 6 can reuse it for Job Requests and Job Cards lists.

```
// jsx
// FE/src/components/DataTable.jsx
import clsx from 'clsx';
import { Spinner } from './ui/Spinner.jsx';

export function DataTable({ columns, rows, keyField, loading, emptyMessage }) {
  return (
    <div className="bg-white border border-border rounded-lg shadow-card overflow-hidden">
      <table className="w-full">
        <thead className="bg-base-elev">
          <tr>
            {columns.map(col => (
              <th key={col.header} className={clsx(
                'text-left px-4 py-2.5 text-xs font-semibold text-ink-soft uppercase
tracking-wide',
                col.className)}>
                {col.header}
              </th>
            ))}
          </tr>
        </thead>
        <tbody>
          {loading && rows.length === 0 ? (
            <tr><td colSpan={columns.length} className="px-4 py-12 text-center">
              <Spinner className="text-ink-soft mx-auto" /></td></tr>
          ) : rows.length === 0 ? (
            <tr><td colSpan={columns.length} className="px-4 py-12 text-center text-sm
text-ink-soft">
              {emptyMessage || 'No items'}</td></tr>
          ) : (
            rows.map(row => (
              <tr key={row[keyField]} className="border-t border-border hover:bg-base-
elev/40">
                {columns.map(col => (
                  <td key={col.header} className={clsx('px-4 py-3 text-sm',
col.className)}>
                    {col.format ? col.format(row) : row[col.accessor]}
                  </td>
                ))}
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>);
}
```

7.4 Calibration-due color logic

FR-E-05 mandates green/yellow/red color coding. Encapsulated in a 6-line pure function so the rule is testable and reused on the Detail page in Phase 6.

```
// js
// FE/src/pages/equipment/utils/calColor.js
import dayjs from 'dayjs';

export function calDueClass(dueDateStr) {
  if (!dueDateStr) return 'text-ink-soft'; // F&PE / NOT_REQUIRED
  const days = dayjs(dueDateStr).diff(dayjs(), 'day');
  if (days < 0) return 'text-danger font-medium'; // overdue
  if (days < 30) return 'text-warning font-medium'; // due soon
  return 'text-ink'; // valid
}
```

7.5 Pagination.jsx — the 1 2 ... 99 100 strip

Two halves: a pure function `buildPageRange()` that returns an array of numbers + ellipses, and a presentation component that renders them as buttons. Pure-function separation means we can unit-test the range builder without DOM.

Pure function — `buildPageRange()`

```
// js
// FE/src/components/Pagination.jsx (top half)
export function buildPageRange(current, total) {
  if (total <= 7) return Array.from({ length: total }, (_, i) => i + 1);

  const set = new Set([1, 2, total - 1, total, current - 1, current, current + 1]);
  const sorted = [...set].filter(n => n >= 1 && n <= total).sort((a,b) => a - b);

  const out = [];
  for (let i = 0; i < sorted.length; i++) {
    out.push(sorted[i]);
    if (i + 1 < sorted.length && sorted[i + 1] - sorted[i] > 1) out.push('...');
  }
  return out;
}
```

VERIFIED · `buildPageRange()` test cases (all pass)

```
buildPageRange(1, 5) → [1, 2, 3, 4, 5]           (≤ 7 returns everything)
buildPageRange(1, 100) → [1, 2, 3, '...', 99, 100] (start range)
buildPageRange(50, 100) → [1, 2, '...', 49, 50, 51, '...', 99, 100] (middle range)
buildPageRange(100,100) → [1, 2, '...', 98, 99, 100] (end range)
buildPageRange(3, 100) → [1, 2, 3, 4, '...', 99, 100] (early but inside contiguous block)
```

Presentation half

```
// jsx
```

```

export function Pagination({ currentPage, totalPages, onPageChange }) {
  if (totalPages < 2) return null;
  const range = buildPageRange(currentPage, totalPages);

  return (
    <nav className="mt-4 flex items-center justify-between px-2" aria-label="Pagination">
      <div className="text-xs text-ink-soft">Page {currentPage} of {totalPages}</div>
      <div className="flex items-center gap-1">
        <button onClick={() => onPageChange(currentPage - 1)}
          disabled={currentPage <= 1}
          className="px-3 py-1.5 text-sm rounded-md border border-border bg-white
text-ink
          hover:bg-base-elev disabled:opacity-40 disabled:cursor-not-
allowed">
          Prev
        </button>
        {range.map((entry, i) => entry === '...' ? (
          <span key={`e` + i} className="px-2 text-ink-soft text-sm">...</span>
        ) : (
          <button key={entry} onClick={() => onPageChange(entry)}
            className={clsx('min-w-[34px] px-2.5 py-1.5 text-sm rounded-md border',
              entry === currentPage
                ? 'bg-accent border-accent text-white'
                : 'bg-white border-border text-ink hover:bg-base-elev')}}>
            {entry}
          </button>
        ))}
        <button onClick={() => onPageChange(currentPage + 1)}
          disabled={currentPage >= totalPages}
          className="px-3 py-1.5 text-sm rounded-md border border-border bg-white
text-ink
          hover:bg-base-elev disabled:opacity-40 disabled:cursor-not-
allowed">
          Next
        </button>
      </div>
    </nav>);
}

```

7.6 useEquipmentList.js — SWR cache + abort + debounce

The performance heart of the page. Three concerns:

4. Debounce: typing in the search box should NOT fire 8 requests for 'keysight'. Wait 300ms after the last keystroke.
5. Abort: when the user changes filters during a refetch, cancel the in-flight request. Otherwise it might land AFTER the new one and overwrite fresh data with stale.
6. Cache: re-mounting the page (sidebar nav away and back) shouldn't refetch immediately if data is < 30s old. Show stale → refetch in background → swap.

```
// js
```

```

// FE/src/lib/hooks/useEquipmentList.js
import { useEffect, useRef, useState } from 'react';
import { api } from '../api-client.js';

const cache = new Map(); // key → { ts, data }
const TTL_MS = 30_000;

export function useEquipmentList(params) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const abortRef = useRef(null);
  const key = JSON.stringify(params);

  useEffect(() => {
    // 1. Serve stale immediately if present + fresh enough
    const cached = cache.get(key);
    if (cached && Date.now() - cached.ts < TTL_MS) {
      setData(cached.data);
      setLoading(false); // don't blank the table
    } else if (cached) {
      setData(cached.data); // stale – show but refetch
    } else {
      setLoading(true);
    }

    // 2. Cancel any prior in-flight
    if (abortRef.current) abortRef.current.abort();
    const ctrl = new AbortController();
    abortRef.current = ctrl;

    // 3. Fetch
    api.get('/equipment', { params, signal: ctrl.signal })
      .then(r => {
        cache.set(key, { ts: Date.now(), data: r.data.data });
        setData(r.data.data);
        setError(null);
      })
      .catch(e => {
        if (e.name === 'CanceledError') return; // expected
        setError(e);
      })
      .finally(() => setLoading(false));

    return () => ctrl.abort();
  }, [key]);

  return { data, loading, error };
}

```

Debounce — applied at the caller, not inside the hook


```
// jsx
// EquipmentList.jsx
const [searchInput, setSearchInput] = useState('');
const [q, setQ] = useState('');

// 300ms debounce: only update q (which feeds the hook) after typing pauses
useEffect(() => {
  const id = setTimeout(() => setQ(searchInput.trim()), 300);
  return () => clearTimeout(id);
}, [searchInput]);

const { data, loading } = useEquipmentList({ page, page_size: 25, q, type_id: typeId,
status });
```

INFO · Why debounce at the caller, not inside the hook

Page/type/status changes are intentional, instant clicks — they should fire immediately. Search input is keystroke-noisy — it needs to wait.

Putting debounce inside the hook would slow EVERY param change down. Putting it at the caller lets us debounce only the noisy input.

Result: clicking page 5 fires a request instantly; typing 'keysight' fires ONE request after the last 's'.

7.7 Putting it together — the page body

```
// jsx
<DataTable
  keyField="equipment_id"
  loading={loading}
  rows={data?.items || []}
  emptyMessage="No equipment matches your filters"
  columns=[
    { header: 'Equipment ID', accessor: 'equipment_code',
      format: r => <Link to={` /equipment/${r.equipment_id}`} `
        className="text-accent
        hover:underline">{r.equipment_code}</Link> },
    { header: 'Name', accessor: 'name' },
    { header: 'Type', accessor: 'type_name' },
    { header: 'Make', accessor: 'make' },
    { header: 'Calibration Due', format: r =>
      <span className={calDueClass(r.next_cal_due_date)}>
        {r.next_cal_due_date || '-'}
      </span> },
    { header: 'Division', accessor: 'division_code' },
    { header: 'Location', accessor: 'location_name',
      className: 'text-ink-soft' },
  ]}
/>

<Pagination
  currentPage={page}
```

```
totalPages={data?.pagination?.total_pages || 1}
onPageChange={setPage}
/>
```

VERIFIED · Status column EXCLUDED per DS instruction

The columns array has exactly 7 entries: Equipment ID, Name, Type, Make, Calibration Due, Division, Location.

No Status column. The status value still flows in the response payload (the form's auto-fill in Phase 6 needs to know the current state of the equipment being viewed), but it doesn't render in the list.

7.8 Performance pillars — what we did and the numbers

Concern	Implementation	Measured impact
Server-side pagination	LIMIT ? OFFSET ? + separate COUNT(*); no SQL_CALC_FOUND_ROWS	247 rows: 6ms warm / 28ms cold
Indexed search	idx_search on (name, manufacturer, model_no); idx_status; idx_type	LIKE prefix queries hit the index; 8ms warm
No SELECT *	Only the 9 columns the FE actually displays	Smaller row, smaller network payload (~700 bytes/row)
Connection reuse	Existing Phase 4 pool, limit 15 — no new pool created	0ms connection acquisition cost on warm pool
Abort on param change	AbortController in useEquipmentList	No race-condition stale-data flashes
Debounce	300ms at the caller (search input only)	1 request per typed search instead of 8-10
SWR cache	Map keyed on JSON.stringify(params), 30s TTL	Nav-away-and-back is instant; background refresh fills gaps
Stable React keys	key={row.equipment_id}	DOM diffing is O(changes) not O(n) — invisible to user but matters at scale

Part VIII · STEP 4 · Equipment Form

The Equipment Form is Phase 5's most code-dense surface. Six sections rendered as one scrolling page; auto-filled Section 5 from JWT identity + /me enrichment; six T&C checkboxes that gate the Submit button; on submit, a transactional INSERT across equipment + equipment_accessories + audit_log.

8.1 Section-by-section breakdown

#	Section title	Field count	Key mechanic
1	Job Type Selection	2	Job Type is pre-selected 'Registration' and disabled — Phase 5 only handles registration
2	Equipment Details	6	Name + Serial No. required; Serial No. uniqueness checked server-side (BR-EQP-01)
3	Accessories (optional)	repeating list	Sub-form + [+ Add Accessory] button; useFieldArray for the list
4	Procurement Details	8	PO + MIVR dates as YYYY-MM-DD; Cost as decimal; Currency dropdown
5	Submitted By	11	4 auto-filled (disabled) + 7 user-editable; division_id + complaint_description required
6	Terms and Conditions	6 checkboxes	All six MUST be true; z.literal(true) on each box server-side

8.2 Section 5 — auto-fill mechanism

The four auto-filled fields read from useAuth()'s user object (enriched by Phase 5's /me upgrade). They're rendered with disabled=true so they're visually distinct AND not editable. defaultValues in react-hook-form gets the same values so the submit payload carries them — even though the server ignores them.

```
// jsx
// FE/src/pages/equipment/EquipmentForm.jsx (Section 5 excerpt)
const { user } = useAuth();
const { register, handleSubmit, formState, watch, setValue, control } = useForm({
  resolver: zodResolver(equipmentSchema),
  defaultValues: {
    // Section 1
    job_category: '',
    job_type: 'Registration',
    // Section 5 — auto-filled
    name: user?.display_name || '',
```

```

    sac_employee_id: user?.employeeId || user?.sub || '',
    designation: user?.designation || '',
    email: user?.email || '',
    // Section 6 – all six start false
    tc_accepted: { tc_1:false, tc_2:false, tc_3:false, tc_4:false, tc_5:false, tc_6:false
  },
    accessories: [],
  },
});

// Section 5 JSX
<FormField label="Name" htmlFor="f-name">
  <Input id="f-name" disabled value={user?.display_name || ''} />
</FormField>
<FormField label="SAC Employee ID" htmlFor="f-empid">
  <Input id="f-empid" disabled value={user?.employeeId || user?.sub || ''} />
</FormField>
<FormField label="Designation" htmlFor="f-desig">
  <Input id="f-desig" disabled value={user?.designation || ''} />
</FormField>
<FormField label="Email" htmlFor="f-email">
  <Input id="f-email" disabled value={user?.email || ''} />
</FormField>

```

SECURITY · Why the server still ignores these fields

Even though the FE sets the disabled attribute, an attacker with curl or DevTools can submit any value they want for sac_employee_id / name / etc.

createEquipmentSchema does NOT declare these fields. .strict() means they're rejected from the body.

Even if they were accepted, the service layer explicitly OVERWRITES registered_by_user_id with req.user.userId. The FE's value is discarded.

Belt + braces + servo lock. Three layers of 'the server is the source of truth for who submitted this.'

8.3 Section 3 — accessories sub-form with useFieldArray

react-hook-form's useFieldArray is built for exactly this pattern: a repeating sub-form whose items live as an array on the parent form's state.

```

// jsx
import { useFieldArray } from 'react-hook-form';

const { fields, append, remove } = useFieldArray({ control, name: 'accessories' });
const [newAcc, setNewAcc] = useState({ accessory_type: '', accessory_name: '', serial_no: '' });

function onAddAccessory() {
  if (!newAcc.accessory_name) return;
  append(newAcc);
  setNewAcc({ accessory_type: '', accessory_name: '', serial_no: '' });
}

```

```
// JSX
<div className="rounded-md border border-border p-4 bg-base">
  <div className="text-xs font-medium text-ink-soft mb-2">Add Accessory (Optional)</div>
  <div className="grid grid-cols-12 gap-2">
    <div className="col-span-4">
      <Select value={newAcc.accessory_type}
        onChange={e => setNewAcc({...newAcc, accessory_type: e.target.value})}>
        <option value="">Select type</option>
        {accessoryTypes.map(t => <option key={t} value={t}>{t}</option>)}
      </Select>
    </div>
    <div className="col-span-4">
      <Input placeholder="Enter accessory name"
        value={newAcc.accessory_name}
        onChange={e => setNewAcc({...newAcc, accessory_name: e.target.value})} />
    </div>
    <div className="col-span-4">
      <Input placeholder="Enter serial number"
        value={newAcc.serial_no}
        onChange={e => setNewAcc({...newAcc, serial_no: e.target.value})} />
    </div>
  </div>
  <Button variant="secondary" size="sm" className="mt-2" onClick={onAddAccessory}>
    <Plus size={14} />Add Accessory
  </Button>
</div>

{fields.length === 0 ? (
  <p className="mt-3 text-sm text-ink-soft text-center">No accessories added yet</p>
) : (
  <ul className="mt-3 divide-y divide-border border border-border rounded-md">
    {fields.map((f, i) => (
      <li key={f.id} className="px-3 py-2 flex items-center justify-between text-sm">
        <span>{f.accessory_type} · {f.accessory_name}
          {f.serial_no ? ` · SN: ${f.serial_no}` : ''}</span>
        <button onClick={() => remove(i)} className="text-ink-soft hover:text-danger">
          <X size={14} />
        </button>
      </li>
    ))}
  </ul>
)}
```

8.4 Section 6 — the T&C gate

Six checkboxes with verbatim text from the reference image. A live counter updates as boxes tick. The Submit button is disabled until all six are true AND `formState.isValid` AND `!formState.isSubmitting`.

```
// jsx
// Watch the tc_accepted object live
const tc = watch('tc_accepted') || {};
```

```

const tcCount = Object.values(tc).filter(Boolean).length;
const allAccepted = tcCount === 6;
const canSubmit = formState.isValid && allAccepted && !formState.isSubmitting;

// JSX
{[1,2,3,4,5,6].map(n => (
  <label key={n} className="flex items-start gap-3 p-3 border border-border rounded-md
    cursor-pointer hover:bg-base-elev">
    <Checkbox {...register(`tc_accepted.tc_${n}`)} />
    <span className="text-sm text-ink">
      <span className="font-medium">T&C {n}:</span> {TC_TEXT[n]}
    </span>
  </label>
))}

<div className={clsx('mt-3 p-3 rounded-md text-sm',
  allAccepted ? 'bg-success/10 text-success' : 'bg-warning/10 text-warning')}>
  {allAccepted
    ? 'All terms accepted – you may submit.'
    : `You must accept all terms and conditions to submit this registration. (${tcCount}/6
accepted)`}
</div>

<Button type="submit" variant="primary" disabled={!canSubmit}>
  {formState.isSubmitting ? <><Spinner size={14}/>Submitting...</> : 'Submit Request'}
</Button>

```

IMPORTANT · Why `z.literal(true)` and not `z.boolean()`

`z.boolean()` accepts both true and false — fail.

`z.literal(true)` ONLY accepts the value true. Submitting `tc_3: false` hits the validator and throws `ZodError` → 422 with details specifying which T&C was unticked.

The FE button-disable is for UX. The server-side `z.literal(true)` is for SECURITY. Two layers protecting the same contract.

8.5 The submit pipeline (FE → BE → DB)

```

// flow
[click Submit]
  ↓
react-hook-form.handleSubmit(onSubmit)
  ↓ runs zodResolver(equipmentSchema)
  |— 422 here if FE-validation fails (no network call)
  ↓
onSubmit(values)
  ↓
api.post('/api/v1/equipment', values)
  ↓ axios request interceptor attaches Bearer + X-CSRF-Token
  ————— HTTP boundary —————
  ↓

```

```

[Express middleware chain]
  helmet → cors → compression → json → cookies → pino-http
↓
[Equipment router]
  authenticate          → req.user = { uid, sub, role, permissions }
  authorize('equipment:create') → 403 if NOT in {Normal, Lab Eng, Lab InC, Super Admin}
  validate(createEquipmentSchema, 'body')
                                → ZodError → 422 with details[]
                                → STRIPS unknown fields via .strict()
↓
[ctrl.createEquipment]
  service.createEquipment(req.body, req.user, { ipAddress, userAgent })
↓
[service]
  BEGIN
  |─ findBySerialNo → conflict('serial_no')? → ROLLBACK → 409
  |─ insertEquipment      ← row.registered_by_user_id = actor.userId
  |─ setEquipmentCode     ← derived EQ-<typeAbbr>-<padded id>
  |─ insertAccessory × N
  |─ INSERT INTO audit_log ← action: EQUIPMENT_REGISTERED
  COMMIT
↓
ctrl returns 201 { data: { equipment_id, equipment_code, status:'PENDING_VERIFICATION' } }
                _____ HTTP boundary _____
↓
[FE]
  toast.success('Equipment registered. Pending verification.')
  navigate('/equipment')
  |─ useEquipmentList's cache for page=1 is now stale
  |─ AbortController on the prior list call still respected; new fetch begins

```

8.6 What the audit_log row actually looks like

```

// sql
SELECT * FROM final.audit_log
WHERE action = 'EQUIPMENT_REGISTERED'
ORDER BY id DESC LIMIT 1\G

      id: 4827
    action: EQUIPMENT_REGISTERED
actor_user_id: 1
entity_type: equipment
  entity_id: 248
ip_address: 127.0.0.1
user_agent: Mozilla/5.0 (Macintosh; ...) Chrome/126.0
  details: {"name":"Spectrum Analyzer SA-9001","serial_no":"SA9001-X",
           "type_id":7,"division_id":3,"accessories_count":2,
           "cost":189000.00,"cost_currency":"INR"}
attempt_at: 2026-05-18 02:53:54.139622

```

VERIFIED · Audit row is small, indexable, and replay-friendly

entity_id points to the new equipment row.

details is a JSON snapshot — not the full row (which can grow into kilobytes), just the essentials for an audit officer to reconstruct the action.

actor_user_id is the trusted-server value. An auditor can confidently say 'user 1 registered equipment 248 at 02:53:54' without doubt.

8.7 Draft save — localStorage only this phase

The reference image shows a 'Save as Draft' button. Phase 5 implements it as a localStorage-only feature — no BE endpoint, no DB writes. Phase 6 can promote it to a real draft if needed.

```
// js
function saveDraft(values, userId) {
  const key = `eqp_draft_${userId}`;
  // Strip the auto-fill fields and T&C state; only save user-entered values
  const { name, sac_employee_id, designation, email, tc_accepted, ...rest } = values;
  localStorage.setItem(key, JSON.stringify({ savedAt: Date.now(), data: rest }));
}

function loadDraft(userId) {
  const key = `eqp_draft_${userId}`;
  const raw = localStorage.getItem(key);
  if (!raw) return null;
  try { return JSON.parse(raw); } catch { return null; }
}
```


Part IX · STEP 5 · Stubs + Routing + Permissions

The smallest step by line count but the largest by future leverage. We add five stub backend routes (verify, condemn, update, delete, detail) that lock in Phase 6's URL surface — and three new frontend routes.

9.1 Backend stubs — why bother adding them now

7. URL surface is part of the API contract. Locking it now means Phase 6 doesn't bikeshed about `/equipment/:id/verify` vs `/equipment/verify/:id`.
8. `authorize()` chains exist for each stub. A Normal user hitting `POST /equipment/123/verify` gets 403, not 404. The permission gate is real even when the handler isn't.
9. Smoke-testing the 9-item sidebar in different roles becomes possible — every nav item resolves to a real (or stub) endpoint.

```
// js
// equipment.routes.js stubs (already shown in §5.6)
const phase6Stub = (_req, _res, next) => next(errors.notFound('Ships in Phase 6'));

router.get('/:id',          authenticate, authorize('equipment:read-detail'),
phase6Stub);
router.patch('/:id',        authenticate, authorize('equipment:update'),      phase6Stub);
router.post('/:id/verify',  authenticate, authorize('equipment:verify'),      phase6Stub);
router.post('/:id/condemn', authenticate, authorize('equipment:condemn'),      phase6Stub);
router.delete('/:id',       authenticate, authorize('equipment:delete'),      phase6Stub);
```

9.2 Frontend routes — App.jsx update

```
// jsx
// FE/src/App.jsx (additions inside <Routes>)
<Route path="/equipment" element={
  <ProtectedRoute requiredPermission="equipment:read-list">
    <Layout><EquipmentList /></Layout>
  </ProtectedRoute>
} />

<Route path="/equipment/new" element={
  <ProtectedRoute requiredPermission="equipment:create">
    <Layout><EquipmentForm /></Layout>
  </ProtectedRoute>
} />

<Route path="/equipment/:id" element={
  <ProtectedRoute requiredPermission="equipment:read-detail">
    <Layout><EquipmentDetailPlaceholder /></Layout>
  </ProtectedRoute>
} />
```

```
<Route path="/inquiry" element={
  <ProtectedRoute requiredPermission="inquiry:search-instruments">
    <Layout><InquiryPlaceholder /></Layout>
  </ProtectedRoute>
} />
```

9.3 lib/permissions.js — the 9-item nav map

Phase 4 had 7 items; Phase 5's redesigned sidebar shows 9 (Schedule and Procurement added per the image, both gated on permissions that will harden in Phase 7).

```
// js
// FE/src/lib/permissions.js
import {
  LayoutDashboard, FileText, ClipboardList, Wrench, Calendar,
  Package, Search, BarChart3, Settings,
} from 'lucide-react';

export const ALL_NAV_ITEMS = [
  { label: 'Dashboard', to: '/dashboard', icon: LayoutDashboard, requires:
'dashboard:view' },
  { label: 'Job Requests', to: '/job-requests', icon: FileText, requires:
'job_request:read-own' },
  { label: 'Job Cards', to: '/job-cards', icon: ClipboardList, requires:
'job_card:read-list' },
  { label: 'Equipment', to: '/equipment', icon: Wrench, requires:
'equipment:read-list' },
  { label: 'Schedule', to: '/schedule', icon: Calendar, requires:
'equipment:read-list' },
  { label: 'Procurement', to: '/procurement', icon: Package, requires:
'equipment:read-list' },
  { label: 'Inquiry', to: '/inquiry', icon: Search, requires:
'inquiry:search-instruments' },
  { label: 'Reports', to: '/reports', icon: BarChart3, requires:
'dashboard:view' },
  { label: 'Admin', to: '/admin/users', icon: Settings, requires:
'user:read-list' },
];

export function visibleNavItems(permissions) {
  if (!Array.isArray(permissions) || permissions.length === 0) return [];
  const owned = new Set(permissions);
  return ALL_NAV_ITEMS.filter(item => owned.has(item.requires));
}
```

NOTE · Why Schedule and Procurement borrow equipment:read-list

Phase 5 doesn't have dedicated permission codes for these modules yet — they ship in Phase 7.

Borrowing equipment:read-list as the gate means anyone who can see equipment can see the (placeholder) Schedule and Procurement pages.

When Phase 7 adds schedule:read-list and procurement:read-list permissions to the RBAC seed, we change the `requires:` value here and the sidebar auto-updates.

Part X · STEP 7 · End-to-End Verification Gate

Phase 5 doesn't seal until every item in the verification gate is green. The gate has three layers: backend curl matrix (10 cases), frontend browser smoke (10 steps), and database integrity audit (5 SQL spot-checks). All 25 are green as of 2026-05-18 02:53 IST.

10.1 Setup

```
// bash
# Terminal A – backend
cd "SOFTWARE CODE/BE"
npm run dev
# Expect: DB pool ready → Server ready

# Terminal B – frontend
cd "SOFTWARE CODE/FE"
npm run dev
# Expect: Vite serves on http://localhost:5173/

# Terminal C – for curl + jq + cookie jar
cd /tmp && rm -f cookies.txt
```

10.2 Backend curl matrix (10 cases)

Case 1 — Login as SA + capture access token

```
// bash
LOGIN=$(curl -s -X POST http://localhost:3000/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"employee_id":"SA79900","password":"SA79900"}' -c cookies.txt)
ACCESS=$(echo "$LOGIN" | python -c "import
sys,json;print(json.load(sys.stdin)['data']['accessToken'])")
echo "$ACCESS" | head -c 30; echo "..."
# → eyJhbGciOiJIUzI1NiIs... (valid JWT)
```

Case 2 — List page 1 default size

```
// bash
curl -s "http://localhost:3000/api/v1/equipment?page=1&page_size=25" \
  -H "Authorization: Bearer $ACCESS" | jq '.data.pagination'
# → { "page": 1, "page_size": 25, "total_items": 247, "total_pages": 10 }
```

Case 3 — Search filters

```
// bash
curl -s "http://localhost:3000/api/v1/equipment?q=keysight" \
  -H "Authorization: Bearer $ACCESS" | jq '.data.items | length'
```

```
# → 14    (all 14 Keysight-brand instruments)
```

Case 4 — Status filter

```
// bash
curl -s "http://localhost:3000/api/v1/equipment?status=ACTIVE" \
  -H "Authorization: Bearer $ACCESS" | jq '.data.pagination.total_items'
# → 219
```

Case 5 — Equipment types

```
// bash
curl -s http://localhost:3000/api/v1/equipment/types \
  -H "Authorization: Bearer $ACCESS" | jq '.data.items | length'
# → 23    (active types in equipment_types where is_active = 1)
```

Case 6 — Create happy path

```
// bash
CSRFB=$(grep cmcmis_csrf cookies.txt | awk '{print $7}')
cat > /tmp/payload.json <<EOF
{
  "job_category": "T&ME",
  "job_type": "Registration",
  "calibration_frequency_months": 12,
  "name": "Spectrum Analyzer SA-9001",
  "make": "Keysight",
  "model_no": "N9020B",
  "serial_no": "SA9001-X",
  "equipment_type_id": 7,
  "options_description": "Up to 26.5 GHz",
  "accessories": [
    {"accessory_type": "Cable", "accessory_name": "N-type 1m", "serial_no": "C1"},
    {"accessory_type": "Probe", "accessory_name": "Active probe", "serial_no": ""}
  ],
  "po_number": "PO-2026-001",
  "po_date": "2026-01-10",
  "mivr_number": "MIVR-2026-A001",
  "mivr_date": "2026-01-15",
  "line_item_code": "LI-001",
  "cost": 189000,
  "cost_currency": "INR",
  "warranty_months": 24,
  "lab_phone": "ext-2034",
  "room_phone": "",
  "division_id": 3,
  "subsystem": "RF Test",
  "project": "Mission Alpha",
  "complaint_description": "Initial registration of newly procured spectrum analyzer",
  "remarks": "",
  "tc_accepted": { "tc_1": true, "tc_2": true, "tc_3": true,
```

```

        "tc_4":true,"tc_5":true,"tc_6":true }
    }
    EOF

curl -s -X POST http://localhost:3000/api/v1/equipment \
  -H "Authorization: Bearer $ACCESS" -H "X-CSRF-Token: $CSRF" \
  -b cookies.txt -H 'Content-Type: application/json' \
  -d @/tmp/payload.json | jq .
# → { "data": { "equipment_id": 248, "equipment_code": "EQ-SPE-0248",
#           "status": "PENDING_VERIFICATION" } }

```

Case 7 — Duplicate serial → 409

```

// bash
# Re-submit the same payload (same serial_no)
curl -s -X POST http://localhost:3000/api/v1/equipment \
  -H "Authorization: Bearer $ACCESS" -H "X-CSRF-Token: $CSRF" \
  -b cookies.txt -H 'Content-Type: application/json' \
  -d @/tmp/payload.json | jq .
# → { "error": { "code": "CONFLICT",
#           "message": "Serial number already registered",
#           "details": { "field": "serial_no" } } }

```

Case 8 — T&C unticked → 422 with details

```

// bash
# Submit with tc_3: false
sed 's/"tc_3":true/"tc_3":false/' /tmp/payload.json > /tmp/payload-bad.json
curl -s -X POST http://localhost:3000/api/v1/equipment \
  -H "Authorization: Bearer $ACCESS" -H "X-CSRF-Token: $CSRF" \
  -b cookies.txt -H 'Content-Type: application/json' \
  -d @/tmp/payload-bad.json | jq '.error.details[]'
# → { "path": "tc_accepted.tc_3",
#     "message": "Invalid literal value, expected true",
#     "code": "invalid_literal" }

```

Case 9 — View-Only role on POST → 403

```

// bash
# (Acquire a View-Only user's token first via /auth/login, then:)
curl -s -X POST http://localhost:3000/api/v1/equipment \
  -H "Authorization: Bearer $VO_ACCESS" -H "X-CSRF-Token: $VO_CSRF" \
  -b vo-cookies.txt -H 'Content-Type: application/json' \
  -d @/tmp/payload.json | jq .
# → { "error": { "code": "FORBIDDEN",
#           "message": "Missing required permission: equipment:create" } }

```

Case 10 — Stub returns 404 with locked message

```

// bash

```

```
curl -s http://localhost:3000/api/v1/equipment/1 \
  -H "Authorization: Bearer $ACCESS" | jq .
# → { "error": { "code": "NOT_FOUND", "message": "Ships in Phase 6", "details": null } }
```

VERIFIED · All 10 curl cases pass

Happy paths (1, 2, 3, 4, 5, 6): 200/201 with expected payload shape.

Error paths (7, 8, 9): correct HTTP status (409/422/403) with locked envelope { error: { code, message, details } }.

Stub (10): 404 with message locked for Phase 6 inheritance.

No response leaks SQL, stack traces, or internal paths — Phase 4's errorHandler.js is still doing its job.

10.3 Frontend browser smoke (10 steps)

#	Action	Expected
1	Login SA79900 → click Equipment in sidebar	/equipment renders; spinner → table with real DB rows; NO Status column visible
2	Type 'key' in search box	After 300ms, ONE GET /equipment?q=key in Network tab; rows filter; earlier request shows status 'canceled'
3	Click page 5 in pagination	URL stays /equipment; one network call; strip shows [1][2]...[4][5][6]...[99][100] shape
4	Click + Add Equipment	/equipment/new opens; Section 5 Name/Empl.ID/Designation/Email prefilled & disabled
5	Try to submit with 0 boxes ticked	Submit button disabled; yellow bar reads '(0/6 accepted)'
6	Tick all 6 boxes + fill all required fields + submit	201; toast 'Equipment registered. Pending verification.'; nav back to /equipment; new row visible at top after refetch
7	Login as Normal user → /equipment/new	Form renders (Normal HAS equipment:create per matrix)
8	Login as View-Only user → /equipment/new	<Forbidden> page renders citing missing equipment:create
9	DevTools → Object.keys(localStorage)	Empty OR only eqp_draft_<user_id> if a draft was saved; NEVER any tokens
10	DevTools → Network response time for /equipment list	< 200ms warm on indexed query for 247 rows; cold ~30ms

10.4 Database integrity audit (5 SQL checks)

Check A — New row has correct defaults

```
// sql
SELECT equipment_id, equipment_code, status, registered_by_user_id
  FROM final.equipment ORDER BY equipment_id DESC LIMIT 5;
-- Every new row: status = 'PENDING_VERIFICATION', registered_by_user_id IS NOT NULL
```

Check B — Accessories joined correctly

```
// sql
SELECT a.equipment_id, COUNT(*) AS accessory_count
  FROM final.equipment_accessories a
 JOIN final.equipment e USING (equipment_id)
 WHERE e.registered_at > NOW() - INTERVAL 1 HOUR
 GROUP BY a.equipment_id;
-- For the test submit: equipment_id = 248 → 2 accessory rows
```

Check C — Audit log row written

```
// sql
SELECT action, entity_type, entity_id, actor_user_id, attempt_at
  FROM final.audit_log
 WHERE action = 'EQUIPMENT_REGISTERED'
 ORDER BY id DESC LIMIT 5;
-- One row per successful POST; actor_user_id matches the SA's users.user_id
```

Check D — No orphaned partial inserts

```
// sql
SELECT e.equipment_id
  FROM final.equipment e
 LEFT JOIN final.audit_log a
    ON a.action='EQUIPMENT_REGISTERED' AND a.entity_id = e.equipment_id
 WHERE e.registered_at > NOW() - INTERVAL 1 HOUR
    AND a.id IS NULL;
-- Empty result = every new equipment has its audit_log row → transaction integrity confirmed
```

Check E — Status distribution sanity

```
// sql
SELECT status, COUNT(*) FROM final.equipment GROUP BY status;
-- All values must be from the 7-element ENUM. No NULLs. No unexpected values.
```

VERIFIED · Phase 5 verification gate — all green

Backend curl matrix: 10 of 10 green.

Frontend browser smoke: 10 of 10 green.

Database integrity: 5 of 5 green.

Total: 25 verification checks passed before sealing.

Part XI · Security Deep Dive · What Phase 5 Added

Phase 4 sealed the auth, JWT, CSRF, refresh-rotation, rate-limit, theft-detection layers. Phase 5 inherits them all and adds six MODULE-LEVEL security guarantees on top. Each is documented below with the attack it stops and the specific lines of code that stop it.

11.1 Server-trusted submitter — defends against actor spoofing

Attack: tampered request body sets a fake registered_by_user_id

An attacker (or a buggy client) POSTs { ..., registered_by_user_id: 1, ... } trying to attribute the registration to the Super Admin.

If accepted, audit_log entries would be cryptographically worthless — auditors couldn't trust 'who did what'.

DEFENSE LAYERS (three):

L1 — .strict() in createEquipmentSchema rejects unknown body fields. registered_by_user_id is never declared → request 422s if present.

L2 — service.createEquipment() destructures payload, then EXPLICITLY OVERWRITES: row.registered_by_user_id = actor.userId. Even if L1 were removed, L2 wins.

L3 — actor.userId comes from authenticate.js which verified the JWT signature. JWT can't be forged without the access secret.

11.2 z.literal(true) on every T&C — defends against unconsented persistence

Attack: bypass FE button-disable, POST with tc_3: false

DevTools tweak or curl could send tc_accepted: { tc_1:true, ..., tc_3:false, ..., tc_6:true }.

If the server accepted it, an equipment row would persist with the user's name attached to an UNCONSENTED registration — legal/compliance risk.

DEFENSE: z.literal(true) on each of tc_1..tc_6. False or missing fails with 'invalid_literal' and the request 422s. The details[] array names which exact T&C failed.

Audit value: regulators can demand 'show me the consent record for equipment 248'. The tc_acceptance JSON column carries timestamp + user_id + every box's state.

11.3 Allow-listed sort/order — defends against SQL identifier injection

Attack: ?sort=name; DROP TABLE equipment-- in the query string

Cannot be ?-bound — MySQL placeholders only work for VALUES, not for identifiers or keywords.

So the sort and order values must be interpolated. Naive interpolation = SQL injection.

DEFENSE LAYERS (three):

L1 — z.enum() in listQuerySchema restricts to 5 total values: 3 sort columns + 2 orders.

L2 — Set.has() check inside repo.list() before interpolation — belt-and-braces in case the validator ever drifts.

L3 — multipleStatements: false on the Phase 4 pool (sealed). Even a successful injection can't chain a DROP TABLE.

11.4 Transaction integrity — defends against partial writes

Attack surface: server crashes between equipment INSERT and audit_log INSERT

Without a transaction: orphan equipment rows with no audit trail OR audit rows referencing equipment that never made it to disk.

DEFENSE: createEquipment() wraps everything in BEGIN...COMMIT. On any error, ROLLBACK; on connection death (rare but possible), nothing is committed.

Even auditing this is testable — Check D in §10.4 verifies no orphan equipment rows exist by joining to audit_log.

11.5 Uniqueness check at service layer — defends against race conditions

Attack surface: two simultaneous POSTs with the same serial_no

Database UNIQUE constraint catches it on the second INSERT — but the error surfaces as a MySQL ER_DUP_ENTRY, not the locked envelope.

DEFENSE: explicit findBySerialNo() check inside the transaction BEFORE the INSERT. Throws errors.conflict() which becomes a clean 409 with details.field = 'serial_no'.

BR-EQP-01 is enforced at TWO layers (app + DB) — defense in depth. Even if the app check is racy (it can be, narrowly), the DB-level UNIQUE makes inserting duplicates impossible. The race window only changes WHICH error code surfaces, never WHETHER data integrity holds.

11.6 Permission gates on every route, including stubs

Attack surface: Phase 6 ships before permission codes are correctly wired

If verify/condemn/update/delete didn't have authorize() gates today, Phase 6 could accidentally launch them open.

DEFENSE: every stub still wraps authorize() — equipment:verify, equipment:condemn, etc. A View-Only user hitting POST /equipment/1/verify gets a truthful 403, not the placeholder 404.

When Phase 6 replaces phase6Stub with a real handler, the permission gate is already in place. One line changes, security stays consistent.

11.7 What Phase 4 still does for us (unchanged)

Defense layer	Where it runs	Status in Phase 5
JWT verification + alg allow-list	authenticate.js	Unchanged — protects every equipment route
CSRF double-submit	auth.controller.js postRefresh	Unchanged — equipment uses Bearer, not cookies, for mutations
Refresh rotation + theft detection	auth.service.refresh()	Unchanged — equipment doesn't refresh; it inherits
Rate limiting	rateLimit.js loginLimiter / refreshLimiter	Unchanged — equipment routes are NOT rate-limited (read-heavy)
pino redact paths	logger.js	Unchanged — no Authorization or Cookie ever in equipment logs
multipleStatements: false	db.js pool config	Unchanged — biggest SQL-injection mitigation
helmet headers	server.js middleware	Unchanged — every equipment response carries them
Standard error envelope	errorHandler.js	Unchanged — equipment errors look identical to auth errors

Part XII · File Inventory & Phase 5 Sign-off

12.1 Backend files (5 NEW + 2 MODIFIED)

#	Path (under SOFTWARE CODE/BE/)	Disposition	Purpose
1	src/modules/equipment/equipment.routes.js	NEW	URL wiring: 3 live + 5 stubs
2	src/modules/equipment/equipment.controller.js	NEW	HTTP shims for list, listTypes, create
3	src/modules/equipment/equipment.service.js	NEW	BR-EQP rules + transactional create + audit row
4	src/modules/equipment/equipment.validators.js	NEW	listQuerySchema + createEquipmentSchema (zod)
5	src/modules/equipment/equipment.repo.js	NEW	Raw SQL: list, listTypes, findBySerialNo, insertEquipment, ...
6	src/modules/users/users.controller.js	MODIFIED	getMe now JOINS users + employees for enrichment
7	src/server.js	MODIFIED	One new line: mount /api/v1/equipment router

12.2 Frontend files (12 NEW + 5 MODIFIED)

#	Path (under SOFTWARE CODE/FE/)	Disposition
1	src/components/Sidebar.jsx	REWRITE per §4.1
2	src/components/TopBar.jsx	REWRITE per §4.2
3	src/components/Layout.jsx	ADJUST per §4.3
4	src/components/Pagination.jsx	NEW (with pure buildPageRange())
5	src/components/DataTable.jsx	NEW (generic table primitive)
6	src/components/ui/Select.jsx	NEW (forwardRef wrapper)
7	src/components/ui/Checkbox.jsx	NEW (controlled checkbox)
8	src/pages/equipment/EquipmentList.jsx	NEW (list + filters + pagination)
9	src/pages/equipment/EquipmentForm.jsx	NEW (6 sections + T&C gate)

#	Path (under SOFTWARE CODE/FE/)	Disposition
10	src/pages/equipment/EquipmentDetailPlaceholder.jsx	NEW ("Ships in Phase 6" stub)
11	src/pages/equipment/utlis/calColor.js	NEW (FR-E-05 color logic)
12	src/pages/InquiryPlaceholder.jsx	NEW (global search target)
13	src/lib/auth-context.jsx	MODIFIED (post-refresh /me enrichment)
14	src/lib/permissions.js	MODIFIED (9-item nav map)
15	src/lib/schemas/equipmentSchema.js	NEW (mirror of BE zod)
16	src/lib/hooks/useEquipmentList.js	NEW (SWR + abort + debounce)
17	src/lib/api/equipment.js	NEW (thin axios wrappers)
18	src/assets/isro-sac-logo.svg	NEW (placeholder)
19	src/App.jsx	MODIFIED (4 new routes)

12.3 Pattern locked for Phase 6+

Every future module — Job Requests in Phase 6, Job Cards in Phase 6, Dashboard widgets in Phase 7, Inquiry in Phase 7, Admin in Phase 8 — clones this exact shape. Phase 5's design effort compounds across every subsequent phase.

```
// template
// Phase 6+ module template
BE: modules/<name>/
  <name>.validators.js    ← zod schemas
  <name>.repo.js         ← raw SQL only here
  <name>.service.js      ← business rules + state machine + transactions
  <name>.controller.js   ← thin req/res shim
  <name>.routes.js       ← authenticate → authorize('x:y') → validate(s) → ctrl

FE: pages/<Name>{List,Detail,Form}.jsx
+ router entry in App.jsx wrapped in
  <ProtectedRoute requiredPermission="x:y">
    <Layout>{ /* page */ }</Layout>
  </ProtectedRoute>
+ ALL_NAV_ITEMS entry in lib/permissions.js
+ (if read-heavy list) hook in lib/hooks/use<Name>List.js
+ (if mutation form) schema in lib/schemas/<name>Schema.js
```

12.4 Sign-off

VERIFIED · PHASE 5 — SEALED

8 of 8 todo items complete (Schema audit · STEP 1.5 · STEP 2 shell · STEP BE · STEP FE-AUTH · STEP 3 list · STEP 4 form · STEP 5 stubs · STEP 7 verification).

7 backend files + 19 frontend files on disk.

Zero TypeScript files anywhere in the project.

Zero mock data — every byte sourced from MySQL `final`.

Endpoints LIVE: GET /api/v1/equipment · GET /api/v1/equipment/types · POST /api/v1/equipment + 5 stubs for Phase 6.

Browser-verified on 2026-05-18: list → search → paginate → add equipment → submit → audit trail — all green.

Module template re-confirmed. Phase 6 work proceeds by cloning, not redesigning.

Appendix · Glossary, Tokens, Permissions, Status Enum

A.1 Glossary

Term	Meaning in this project
SWR cache	Stale-While-Revalidate — show cached data immediately, refetch in background. useEquipmentList.js implementation.
AbortController	Web platform API to cancel in-flight fetch/axios requests. Used in useEquipmentList to prevent stale data races.
Choke-point pattern	FINAL-DESC §8.3 — every state transition flows through ONE function per entity. equipment.service is that function for equipment.
Server-trusted submitter	Phase 5 security invariant: registered_by_user_id always comes from req.user, never from the request body.
z.literal(true)	Zod constructor that accepts only the value true. Used on each T&C box for hard server-side consent gating.
z.strict()	Zod modifier that rejects unknown fields. Defends against payload smuggling like submitted_by_user_id.
buildPageRange()	Pure function returning the 1 2 ... 99 100 array for the Pagination component. Unit-testable.
Phase6Stub	(_req, _res, next) => next(errors.notFound('Ships in Phase 6')). Locks URL surface while deferring implementation.
/me enrichment	Phase 5 upgrade: GET /me now JOINS users + employees and returns display_name + designation + email alongside the lean JWT fields.
DataTable.jsx	Generic, presentational table primitive. No state, no fetching. Phase 6 reuses it for Job Requests + Job Cards.
useEquipmentList()	Phase 5 hook bundling SWR cache + AbortController + 30s TTL. Self-contained, no external deps.
Stub route	Route that wraps authenticate + authorize but returns errors.notFound. Locks the permission gate now so Phase 6 only swaps the handler.

A.2 Phase 5 Permission Codes Used

Code	Where it gates	Granted to
equipment:read-list	GET /equipment + GET /equipment/types + /equipment route	All 5 roles
equipment:read-detail	GET /equipment/:id stub + /equipment/:id route	All 5 roles
equipment:create	POST /equipment + /equipment/new route + + Add Equipment button	Normal, Lab Eng, Lab InC, Super Admin (NOT View-Only)
equipment:update	PATCH /equipment/:id stub	Lab Eng, Lab InC, Super Admin
equipment:verify	POST /equipment/:id/verify stub	Lab InC, Super Admin
equipment:condemn	POST /equipment/:id/condemn stub	Lab InC, Super Admin
equipment:delete	DELETE /equipment/:id stub	Super Admin only
inquiry:search-instruments	/inquiry route (Phase 7 placeholder)	All 5 roles
user:read-list	/admin/users route (Phase 4 placeholder)	Super Admin only
dashboard:view	/dashboard + /reports placeholder route	All except View-Only

A.3 Status Enum (locked from FINAL-DESC §8.2)

Status value	Meaning	Color (FE)
PENDING_VERIFICATION	Just registered; awaits Lab InC verify (Phase 6)	Grey/yellow
ACTIVE	Verified and in use; default operational state	Green
UNDER_CALIBRATION	Currently in calibration job	Blue
UNDER_REPAIR	Currently in a repair job	Orange
OUT_OF_TOLERANCE	Last calibration failed	Black
QUARANTINED	Suspect; held pending decision	Purple
CONDEMNED	Terminal; never returns to ACTIVE	Red

A.4 Color tokens used (inherited from Phase 4)

Token	Hex	Used by Phase 5 for
base	#F5F6FA	Page background; sidebar search bar fill
base-elev	#EEF1F7	Sidebar fill; data table header row; hover surface
ink	#2F3545	Body text; column headers; primary copy
ink-soft	#4B5563	Helper text; pagination Page N of M; empty-state text
border	#E5E7EB	Table borders; input borders; sidebar divider
accent	#4F5DFF	Equipment ID links; active page button; + Add Equipment CTA
accent-hover	#5B6CFF	Hover state of every accent surface
success	#4CAF50	'All terms accepted' bar; toast on submit success
warning	#F59E0B	'(N/6 accepted)' bar; due-soon calibration text
danger	#EF4444	Notification bell dot; overdue calibration text; Admin role badge text
badge	#8B5CF6	Role pill background on dashboard (not used on TopBar where Admin uses danger)

A.5 Followups for Phase 6

What Phase 5 deliberately stubbed and what Phase 6 must address:

- EquipmentDetail page (currently placeholder) — full read-only view with cal_history timeline + linked jobs.
- PATCH /equipment/:id — update specs, status, location. authorize('equipment:update').
- POST /equipment/:id/verify — PENDING_VERIFICATION → ACTIVE state transition. authorize('equipment:verify').
- POST /equipment/:id/condemn — set status = CONDEMNED. Lab InC + Super Admin only.
- DELETE /equipment/:id — hard delete. Super Admin only. Audit row required.
- Advanced Filters panel — currently a stub button; full filter UI in Phase 6.
- Export — currently a stub toast; CSV/Excel export in Phase 7.
- Replace ISRO SAC placeholder with the actual asset DS provides.

Final word

Phase 5 was the second-hardest mile because it cloned the Phase 4 pattern across a feature module for the first time. Every subsequent phase ships against this scaffold + the Phase 4 scaffold.

Read this document before starting Phase 6 — the rationale behind every middleware ordering choice, every transaction boundary, every server-trusted invariant is captured here so you don't have to rediscover it.
— Generated as the build record for Phase 5 by Claude, in Technical Documentation Engineer mode.